Machine Learning 1: 81–106, 1986
© 1986 Kluwer Academic Publishers, Boston – Manufactured in The Netherlands

Induction of Decision Trees

J.R. QUINLAN (munnari!nswitgould.oz!quinlan@seismo.css.gov)
Centre for Advanced Computing Sciences, New South Wales Institute of Technology, Sydney 2007, Australia

(Received August 1, 1985)

Key words: classification, induction, decision trees, information theory, knowledge acquisition, expert systems

Abstract. The technology for building knowledge-based systems by inductive inference from examples has been demonstrated successfully in several practical applications. This paper summarizes an approach to synthesizing decision trees that has been used in a variety of systems, and it describes one such system, ID3, in detail. Results from recent studies show ways in which the methodology can be modified to deal with information that is noisy and/or incomplete. A reported shortcoming of the basic algorithm is discussed and two means of overcoming it are compared. The paper concludes with illustrations of current research directions.

1. Introduction

Since artificial intelligence first achieved recognition as a discipline in the mid 1950's, machine learning has been a central research area. Two reasons can be given for this prominence. The ability to learn is a hallmark of intelligent behavior, so any attempt to understand intelligence as a phenomenon must include an understanding of learning. More concretely, learning provides a potential methodology for building high-performance systems.

Research on learning is made up of diverse subfields. At one extreme there are adaptive systems that monitor their own performance and attempt to improve it by adjusting internal parameters. This approach, characteristic of a large proportion of the early learning work, produced self-improving programs for playing games (Samuel, 1967), balancing poles (Michie, 1982), solving problems (Quinlan, 1969) and many other domains. A quite different approach sees learning as the acquisition of structured knowledge in the form of concepts (Hunt, 1962; Winston, 1975), discrimination nets (Feigenbaum and Simon, 1963), or production rules (Buchanan, 1978).

The practical importance of machine learning of this latter kind has been underlin-

Machine Learning 1: 81-106, 1986 © 1986 Kluwer Academic Publishers, Boston - Fabricado na Holanda

Indução de Árvores de Decisão

JR QUINLAN (munnari! nswitgould.oz! quinlan@ seismo.css.gov) Centro de Ciências da Computação Avançada, Instituto de Tecnologia de Nova Gales do Sul, Sydney 2007, Austrália

(Recebido em 1º de agosto de 1985)

Palavras-chave: classificação, indução, árvores de decisão, teoria da informação, aquisição de conhecimento, sistemas especialistas

Resumo. A tecnologia para construção de sistemas baseados em conhecimento por inferência indutiva a partir de exemplos foi demonstrada com sucesso em diversas aplicações práticas. Este artigo resume uma abordagem para sintetizar árvores de decisão que tem sido usada em uma variedade de sistemas e descreve detalhadamente um desses sistemas, o ID3. Os resultados de estudos recentes mostram maneiras pelas quais a metodologia pode ser modificada para lidar com informações ruidosas e/ou incompletas. Uma deficiência relatada no algoritmo básico é discutida e duas maneiras de superá-la são comparadas. O artigo conclui com ilustrações das direções atuais da pesquisa.

1. Introdução

Desde que a inteligência artificial alcançou o reconhecimento como disciplina pela primeira vez em meados da década de 1950, a aprendizagem automática tem sido uma área central de investigação. Duas razões podem ser dadas para esta proeminência. A capacidade de aprender é uma marca distintiva do comportamento inteligente, por isso qualquer tentativa de compreender a inteligência como um fenómeno deve incluir uma compreensão da aprendizagem. Mais concretamente, a aprendizagem proporciona uma metodologia potencial para a construção de sistemas de alto desempenho.

A pesquisa sobre aprendizagem é composta por diversos subcampos. Num extremo estão os sistemas adaptativos que monitorizam o seu próprio desempenho e tentam melhorá-lo ajustando parâmetros internos. Esta abordagem, característica de grande parte do trabalho de aprendizagem inicial, produziu programas de auto-aperfeiçoamento para jogar jogos (Samuel, 1967), equilibrar postes (Michie, 1982), resolver problemas (Quinlan, 1969) e muitos outros domínios. Uma abordagem bastante diferente vê a aprendizagem como a aquisição de conhecimento estruturado na forma de conceitos (Hunt, 1962; Winston, 1975), redes de discriminação (Feigenbaum e Simon, 1963) ou regras de produção (Buchanan, 1978).

A importância prática do aprendizado de máquina deste último tipo tem sido sublinhada

ed by the advent of knowledge-based expert systems. As their name suggests, these systems are powered by knowledge that is represented explicitly rather than being implicit in algorithms. The knowledge needed to drive the pioneering expert systems was codified through protracted interaction between a domain specialist and a knowledge engineer. While the typical rate of knowledge elucidation by this method is a few rules per man day, an expert system for a complex task may require hundreds or even thousands of such rules. It is obvious that the interview approach to knowledge acquisition cannot keep pace with the burgeoning demand for expert systems; Feigenbaum (1981) terms this the 'bottleneck' problem. This perception has stimulated the investigation of machine learning methods as a means of explicating knowledge (Michie, 1983).

This paper focusses on one microcosm of machine learning and on a family of learning systems that have been used to build knowledge-based systems of a simple kind. Section 2 outlines the features of this family and introduces its members. All these systems address the same task of inducing decision trees from examples. After a more complete specification of this task, one system (ID3) is described in detail in Section 4. Sections 5 and 6 present extensions to ID3 that enable it to cope with noisy and incomplete information. A review of a central facet of the induction algorithm reveals possible improvements that are set out in Section 7. The paper concludes with two novel initiatives that give some idea of the directions in which the family may grow.

2. The TDIDT family of learning systems

Carbonell, Michalski and Mitchell (1983) identify three principal dimensions along which machine learning systems can be classified:

- the underlying learning strategies used;
- the representation of knowledge acquired by the system; and
- the application domain of the system.

This paper is concerned with a family of learning systems that have strong common bonds in these dimensions.

Taking these features in reverse order, the *application domain* of these systems is not limited to any particular area of intellectual activity such as Chemistry or Chess; they can be applied to any such area. While they are thus general-purpose systems, the applications that they address all involve *classification*. The product of learning is a piece of procedural knowledge that can assign a hitherto-unseen object to one of a specified number of disjoint classes. Examples of classification tasks are:

ed pelo advento de sistemas especialistas baseados em conhecimento. Como o próprio nome sugere, esses sistemas são alimentados por conhecimento que é representado explicitamente, em vez de estar implícito em algoritmos. O conhecimento necessário para impulsionar os sistemas especialistas pioneiros foi codificado através de uma interação prolongada entre um especialista do domínio e um engenheiro do conhecimento. Embora a taxa típica de elucidação do conhecimento por este método seja de algumas regras por homem/dia, um sistema especialista para uma tarefa complexa pode exigir centenas ou mesmo milhares de tais regras. É óbvio que a abordagem de entrevista para a aquisição de conhecimento não consegue acompanhar a crescente procura de sistemas especialistas; Feigenbaum (1981) chama isso de problema do "gargalo". Essa percepção estimulou a investigação de métodos de aprendizado de máquina como meio de explicar o conhecimento (Michie, 1983).

Este artigo se concentra em um microcosmo de aprendizado de máquina e em uma família de sistemas de aprendizado que têm sido usados para construir siste mas simples baseados em conhecimento. A seção 2 descreve as características desta família e apresenta seus membros. Todos estes sistemas abordam a mesma tarefa de induzir árvores de decisão a partir de exemplos. Após uma especificação mais completa desta tarefa, um sistema (ID3) é descrito em detalhes na Seção 4. As Seções 5 e 6 apresentam extensões ao ID3 que o permitem lidar com informações ruidosas e incompletas. Uma revisão de uma faceta central do algoritmo de indução revela possíveis melhorias que são apresentadas na Secção 7. O artigo conclui com duas novas iniciativas que dão uma ideia das direcções em que a família pode crescer.

2. A família TDIDT de sistemas de aprendizagem

Carbonell, Michalski e Mitchell (1983) identificam três dimensões principais ao longo das quais os sistemas de aprendizado de máquina podem ser classificados:

- as estratégias de aprendizagem subjacentes utilizadas;
- a representação do conhecimento adquirido pelo sistema; e
- o domínio do aplicativo do sistema.

Este artigo trata de uma família de sistemas de aprendizagem que possuem fortes laços comuns nessas dimensões.

Tomando estas características na ordem inversa, o domínio de aplicação destes sistemas não se limita a nenhuma área particular da atividade intelectual como a Química ou o Xadrez; eles podem ser aplicados a qualquer área desse tipo. Embora sejam sistemas de uso geral, todas as aplicações que eles abordam envolvem classificação. O produto da aprendizagem é uma parte do conhecimento processual que pode atribuir um objeto até então não visto a uma de um número específico de classes disjuntas. Exemplos de tarefas de classificação são:

1. the diagnosis of a medical condition from symptoms, in which the classes could be either the various disease states or the possible therapies;
2. determining the game-theoretic value of a chess position, with the classes *won for white*, *lost for white*, and *drawn*; and
3. deciding from atmospheric observations whether a severe thunderstorm is unlikely, possible or probable.

It might appear that classification tasks are only a minuscule subset of procedural tasks, but even activities such as robot planning can be recast as classification problems (Dechter and Michie, 1985).

The members of this family are sharply characterized by their *representation of acquired knowledge* as decision trees. This is a relatively simple knowledge formalism that lacks the expressive power of semantic networks or other first-order representations. As a consequence of this simplicity, the learning methodologies used in the TDIDT family are considerably less complex than those employed in systems that can express the results of their learning in a more powerful language. Nevertheless, it is still possible to generate knowledge in the form of decision trees that is capable of solving difficult problems of practical significance.

The *underlying strategy* is non-incremental learning from examples. The systems are presented with a set of cases relevant to a classification task and develop a decision tree from the top down, guided by frequency information in the examples but not by the particular order in which the examples are given. This contrasts with incremental methods such as that employed in MARVIN (Sammut, 1985), in which a dialog is carried on with an instructor to 'debug' partially correct concepts, and that used by Winston (1975), in which examples are analyzed one at a time, each producing a small change in the developing concept; in both of these systems, the order in which examples are presented is most important. The systems described here search for patterns in the given examples and so must be able to examine and re-examine all of them at many stages during learning. Other well-known programs that share this data-driven approach include BACON (Langley, Bradshaw and Simon, 1983) and INDUCE (Michalski, 1980).

In summary, then, the systems described here develop decision trees for classification tasks. These trees are constructed beginning with the root of the tree and proceeding down to its leaves. The family's palindromic name emphasizes that its members carry out the *T*op-*D*own *I*nduction of *D*ecision *T*rees.

The example objects from which a classification rule is developed are known only through their values of a set of properties or attributes, and the decision trees in turn are expressed in terms of these same attributes. The examples themselves can be assembled in two ways. They might come from an existing database that forms a history of observations, such as patient records in some area of medicine that have accumulated at a diagnosis center. Objects of this kind give a reliable statistical picture but, since they are not organized in any way, they may be redundant or omit

1. o diagnóstico de uma condição médica a partir de sintomas, em que as classes podem ser os vários estados de doença ou as terapias possíveis; 2. determinar o valor teórico do jogo de uma posição de xadrez, com as classes vencidas para as brancas, perdidas para as brancas e empatadas; e 3. decidir, a partir de observações atmosféricas, se uma tempestade severa é improvável, possível ou provável.

Pode parecer que as tarefas de classificação são apenas um subconjunto minúsculo de tarefas processuais, mas mesmo actividades como o planeamento de robôs podem ser reformuladas como problemas de classificação (Dechter e Michie, 1985).

Os membros desta família são claramente caracterizados pela representação do conhecimento adquirido como árvores de decisão. Este é um formalismo de conhecimento relativamente simples que carece do poder expressivo das redes semânticas ou de outras representações de primeira ordem. Como consequência desta simplicidade, as metodologias de aprendizagem utilizadas na família T D I D T são consideravelmente menos complexas do que aquelas empregadas em sistemas que podem expressar os resultados da sua aprendizagem numa linguagem mais poderosa. No entanto, ainda é possível gerar conhecimento na forma de árvores de decisão capazes de resolver problemas difíceis e de importância prática.

A estratégia subjacente é a aprendizagem não incremental a partir de exemplos. Os sistemas são apresentados a um conjunto de casos relevantes para uma tarefa de classificação e desenvolvem uma árvore de decisão de cima para baixo, guiada pelas informações de frequência nos exemplos, mas não pela ordem específica em que os exemplos são dados. Isto contrasta com métodos incrementais como o empregado em MARVIN (Sammut, 1985), no qual um diálogo é mantido com um instrutor para " depurar" conceitos parcialmente corretos, e aquele usado por Winston (1975), no qual os exemplos são analisados um de cada vez, cada um produzindo uma pequena mudança no conceito em desenvolvimento; em ambos os sistemas, a ordem em que os exemplos são apresentados é mais importante. Os sistemas aqui descritos procuram padrões nos exemplos dados e, portanto, devem ser capazes de examinar e reexaminar todos eles em vários estágios durante a aprendizagem. Outros progra mas bem conhecidos que partilham esta abordagem baseada em dados incluem BACON (Langley, Bradshaw e Simon, 1983) e INDUCE (Michalski, 1980).

Em resumo, então, os sistemas aqui descritos desenvolvem árvores de decisão para tarefas de classificação. Estas árvores são construídas começando pela raiz da árvore e seguindo até as folhas. O nome palíndromo da família enfatiza que seus membros realizam a Indução Top-Down de Árvores de Decisão.

Os objetos de exemplo a partir dos quais uma regra de classificação é desenvolvida são conhecidos apenas através dos valores de um conjunto de propriedades ou atributos, e as árvores de decisão, por sua vez, são expressas em termos desses mesmos atributos. Os próprios exemplos podem ser montados de duas maneiras. Eles podem vir de um banco de dados existente que forma um histórico de observações, como registros de pacientes em alguma área da medicina que foram acumulados em um centro de diagnóstico. Objectos deste tipo fornecem uma imagem estatística fiável mas, como não estão organizados de forma alguma, podem ser redundantes ou omitir dados.

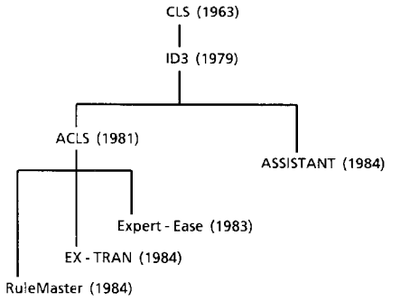


Figure 1. The TDIDT family tree.

uncommon cases that have not been encountered during the period of record-keeping. On the other hand, the objects might be a carefully culled set of tutorial examples prepared by a domain expert, each with some particular relevance to a complete and correct classification rule. The expert might take pains to avoid redundancy and to include examples of rare cases. While the family of systems will deal with collections of either kind in a satisfactory way, it should be mentioned that earlier TDIDT systems were designed with the 'historical record' approach in mind, but all systems described here are now often used with tutorial sets (Michie, 1985).

Figure 1 shows a family tree of the TDIDT systems. The patriarch of this family is Hunt's Concept Learning System framework (Hunt, Marin and Stone, 1966). CLS constructs a decision tree that attempts to minimize the cost of classifying an object. This cost has components of two types: the measurement cost of determining the value of property A exhibited by the object, and the misclassification cost of deciding that the object belongs to class J when its real class is K. CLS uses a lookahead strategy similar to minimax. At each stage, CLS explores the space of possible decision trees to a fixed depth, chooses an action to minimize cost in this limited space, then moves one level down in the tree. Depending on the depth of lookahead chosen, CLS can require a substantial amount of computation, but has been able to unearth subtle patterns in the objects shown to it.

ID3 (Quinlan, 1979, 1983a) is one of a series of programs developed from CLS in response to a challenging induction task posed by Donald Michie, viz. to decide from pattern-based features alone whether a particular chess position in the King-Rook vs King-Knight endgame is lost for the Knight's side in a fixed number of ply. A full description of ID3 appears in Section 4, so it is sufficient to note here that it embeds a tree-building method in an iterative outer shell, and abandons the cost-driven lookahead of CLS with an information-driven evaluation function.

ACLS (Paterson and Niblett, 1983) is a generalization of ID3. CLS and ID3 both require that each property used to describe objects has only values from a specified set. In addition to properties of this type, ACLS permits properties that have

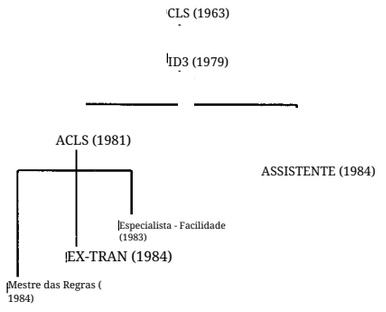


Figura 1. A árvore genealógica do TDIDT.

casos incomuns que não foram encontrados durante o período de manutenção de registros. Por outro lado, os objetos podem ser um conjunto cuidadosamente selecionado de exemplos tutoriais preparados por um especialista no domínio, cada u m com alguma relevância particular para uma regra de classificação completa e correta. O especialista poderá esforçar-se por evitar redundâncias e incluir exemplos de casos raros. Embora a família de sistemas lide com coleções de qualquer tipo de maneira satisfatória, deve ser mencionado que os sistemas T D I D T anteriores foram projetados com a abordagem de "registro histórico" em mente, mas todos os sistemas aqui descritos são agora frequentemente usados com conjuntos tutoriais (Michie, 1985).

A Figura 1 mostra uma árvore genealógica dos sistemas T D I D T. O patriarca desta família é a estrutura do Concept Learning System de Hunt (Hunt, Marin e Stone, 1966). O CLS constrói uma árvore de decisão que tenta minimizar o custo de classificação de um objeto. Este custo tem componentes de dois tipos: o custo de medição para determinar o valor da propriedade A exibida pelo objeto, e o custo de classificação incorreta para decidir que o objeto pertence à classe J quando sua classe real é K. O CLS usa uma estratégia de antecipação semelhante ao minimax. Em cada estágio, o CLS explora o espaço de possíveis árvores de decisão até uma profundidade fixa, escolhe uma ação para minimizar o custo neste espaço limitado e, em seguida, move-se um nível para baixo na árvore. Dependendo da profundidade de antecipação escolhida, o CLS pode exigir uma quantidade substancial de computação, mas foi capaz de descobrir padrões sutis nos objetos mostrados a ele.

ID3 (Quinlan, 1979, 1983a) é um de uma série de programas desenvolvidos a partir do CLS em resposta a uma desafiadora tarefa de indução apresentada por Donald Michie, viz. para decidir apenas a partir de recursos baseados em padrões se uma determinada posição de xadrez no final do jogo Rei-Torre vs Rei-Cavaleiro é perdida para o lado do Cavalo em um número fixo de jogadas. Uma descrição completa do ID3 aparece na Seção 4, portanto é suficiente observar aqui que ele incorpora um método de construção de árvore em um shell externo iterativo e abandona a previsão orientada por custos do CLS com uma função de avaliação orientada por informações.

ACLS (Paterson e Niblett, 1983) é uma generalização do ID3. CLS e ID3 exigem que cada propriedade usada para descrever objetos tenha apenas valores de um conjunto especificado. Além de propriedades deste tipo, o ACLS permite propriedades que tenham

unrestricted integer values. The capacity to deal with attributes of this kind has allowed ACLS to be applied to difficult tasks such as image recognition (Shepherd, 1983).

ASSISTANT (Kononenko, Bratko and Roskar, 1984) also acknowledges ID3 as its direct ancestor. It differs from ID3 in many ways, some of which are discussed in detail in later sections. ASSISTANT further generalizes on the integer-valued attributes of ACLS by permitting attributes with continuous (real) values. Rather than insisting that the classes be disjoint, ASSISTANT allows them to form a hierarchy, so that one class may be a finer division of another. ASSISTANT does not form a decision tree iteratively in the manner of ID3, but does include algorithms for choosing a 'good' training set from the objects available. ASSISTANT has been used in several medical domains with promising results.

The bottom-most three systems in the figure are commercial derivatives of ACLS. While they do not significantly advance the underlying theory, they incorporate many user-friendly innovations and utilities that expedite the task of generating and using decision trees. They all have industrial successes to their credit. Westinghouse Electric's Water Reactor Division, for example, points to a fuel-enrichment application in which the company was able to boost revenue by 'more than ten million dollars per annum' through the use of one of them.[1]

3. The induction task

We now give a more precise statement of the induction task. The basis is a universe of *objects* that are described in terms of a collection of *attributes*. Each attribute measures some important feature of an object and will be limited here to taking a (usually small) set of discrete, mutually exclusive values. For example, if the objects were Saturday mornings and the classification task involved the weather, attributes might be

outlook, with values {sunny, overcast, rain}
temperature, with values {cool, mild, hot}
humidity, with values {high, normal}
windy, with values {true, false}

Taken together, the attributes provide a zeroth-order language for characterizing objects in the universe. A particular Saturday morning might be described as

outlook: overcast
temperature: cool
humidity: normal
windy: false

[1] Letter cited in the journal *Expert Systems* (January, 1985), p. 20.

valores inteiros irrestritos. A capacidade de lidar com atributos deste tipo permitiu que o ACLS fosse aplicado a tarefas difíceis como o reconhecimento de imagens (Shepherd, 1983).

ASSISTANT (Kononenko, Bratko e Roskar, 1984) também reconhece o ID3 como seu ancestral direto. Ele difere do ID3 de várias maneiras, algumas das quais serão discutidas em detalhes nas seções posteriores. O ASSISTANT generaliza ainda mais os atributos com valor inteiro do ACLS, permitindo atributos com valores contínuos (reais). Em vez de insistir que as classes sejam disjuntas, o ASSISTANT permite que elas formem uma hierarquia, de modo que uma classe possa ser uma divisão mais refinada de outra. O ASSISTANT não forma uma árvore de decisão iterativamente como o ID3, mas inclui algoritmos para escolher um 'bom' conjunto de treinamento a partir dos objetos disponíveis. O ASSISTANT tem sido utilizado em diversas áreas médicas com resultados promissores.

Os três sistemas mais abaixo na figura são derivados comerciais do ACLS. Embora não avancem significativamente a teoria subjacente, incorporam muitas inovações e utilidades fáceis de usar que agilizam a tarefa de gerar e usar árvores de decisão. Todos eles têm sucessos industriais em seu crédito. A Divisão de Reatores de Água da Westinghouse Electric, por exemplo, aponta para uma aplicação de enriquecimento de combustível na qual a empresa conseguiu aumentar a receita em " mais de dez milhões de dólares por ano" através do uso de um deles. 1

3. A tarefa de indução

Damos agora uma declaração mais precisa da tarefa de indução. A base é um universo de objetos descritos em termos de uma coleção de atributos. Cada atributo mede alguma característica importante de um objeto e será limitado aqui a assumir um conjunto (geralmente pequeno) de valores discretos e mutuamente exclusivos. Por exemplo, se os objetos fossem manhãs de sábado e a tarefa de classificação envolvesse o clima, os atributos poderiam ser

perspectiva, com valores {ensolarado, nublado,
chuva]temperatura, com valores {frio, ameno,
quente]umidade, com valores {alto, normal]
vento, com valores {verdadeiro, falso]

Tomados em conjunto, os atributos fornecem uma linguagem de ordem zero para caracterizar objetos no universo. Uma determinada manhã de sábado pode ser descrita como

perspectiva:
nublado
temperatura: fresco
umidade: normal
ventoso: falso

1 Carta citada na revista Expert Systems (janeiro de 1985), p. 20.

Each object in the universe belongs to one of a set of mutually exclusive *classes*. To simplify the following treatment, we will assume that there are only two such classes denoted *P* and *N*, although the extension to any number of classes is not difficult. In two-class induction tasks, objects of class P and N are sometimes referred to as *positive instances* and *negative instances*, respectively, of the concept being learned.

The other major ingredient is a *training set* of objects whose class is known. The induction task is to develop a *classification rule* that can determine the class of any object from its values of the attributes. The immediate question is whether or not the attributes provide sufficient information to do this. In particular, if the training set contains two objects that have identical values for each attribute and yet belong to different classes, it is clearly impossible to differentiate between these objects with reference only to the given attributes. In such a case attributes will be termed *inadequate* for the training set and hence for the induction task.

As mentioned above, a classification rule will be expressed as a decision tree. Table 1 shows a small training set that uses the 'Saturday morning' attributes. Each object's value of each attribute is shown, together with the class of the object (here, class P mornings are suitable for some unspecified activity). A decision tree that correctly classifies each object in the training set is given in Figure 2. Leaves of a decision tree are class names, other nodes represent attribute-based tests with a branch for each possible outcome. In order to classify an object, we start at the root of the tree, evaluate the test, and take the branch appropriate to the outcome. The process continues until a leaf is encountered, at which time the object is asserted to belong to

Table 1. A small training set

No.	Attributes				Class
	Outlook	Temperature	Humidity	Windy	
1	sunny	hot	high	false	N
2	sunny	hot	high	true	N
3	overcast	hot	high	false	P
4	rain	mild	high	false	P
5	rain	cool	normal	false	P
6	rain	cool	normal	true	N
7	overcast	cool	normal	true	P
8	sunny	mild	high	false	N
9	sunny	cool	normal	false	P
10	rain	mild	normal	false	P
11	sunny	mild	normal	true	P
12	overcast	mild	high	true	P
13	overcast	hot	normal	false	P
14	rain	mild	high	true	N

Cada objeto no universo pertence a um conjunto de classes mutuamente exclusivas. Para simplificar o tratamento a seguir, assumiremos que existem apenas duas dessas classes denotadas P e N, embora a extensão para qualquer número de classes não seja difícil. Em tarefas de indução de duas classes, os objetos das classes P e N são às vezes chamados de instâncias positivas e instâncias negativas, respectivamente, do conceito que está sendo aprendido.

O outro ingrediente principal é um conjunto de treinamento de objetos cuja classe é conhecida. A tarefa de indução é desenvolver uma regra de classificação que possa determinar a classe de qualquer objeto a partir dos valores dos atributos. A questão imediata é se os atributos fornecem ou não informações suficientes para fazer isso. Em particular, se o conjunto de treinamento contém dois objetos que possuem valores idênticos para cada atributo e ainda pertencem a classes diferentes, é claramente impossível diferenciar entre esses objetos com referência apenas aos atributos dados. Nesse caso, os atributos serão considerados inadequados para o conjunto de treinamento e, portanto, para a tarefa de indução.

Conforme mencionado acima, uma regra de classificação será expressa como uma árvore de decisão. A Tabela 1 mostra um pequeno conjunto de treinamento que utiliza os atributos 'Sábado de manhã'. O valor de cada atributo de cada objeto é mostrado, juntamente com a classe do objeto (aqui, as manhãs da classe P são adequadas para alguma atividade não especificada). Uma árvore de decisão que classifica corretamente cada objeto no conjunto de treinamento é fornecida na Figura 2. As folhas de uma árvore de decisão são nomes de classes, outros nós representam testes baseados em atributos com uma ramificação para cada res ultado possível. Para classificar um objeto, começamos na raiz da árvore, avaliamos o teste e pegamos o ramo apropriado ao resultado. O processo continua até que uma folha seja encontrada, momento em que o objeto é declarado pertencer a ela.

Tabela 1. Um pequeno conjunto de treinamento

	Atributos				Aula
	Panorama	Temperatura	Umidade	Ventoso	
	ensolarado			falso	
	ensolarado				
	nublado			falso	
				falso	
			normal	falso	
			normal		
	nublado		normal		
	ensolarado			falso	
	ensolarado		normal	falso	
			normal	falso	
	ensolarado		normal		
	nublado				
	nublado		normal	falso	

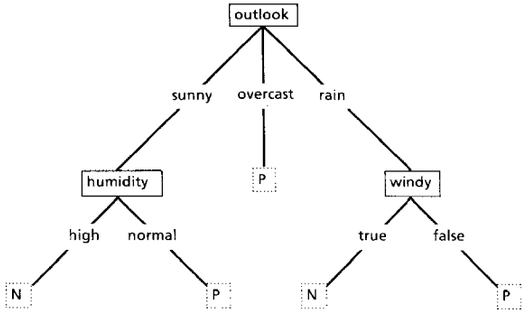


Figure 2. A simple decision tree

the class named by the leaf. Taking the decision tree of Figure 2, this process concludes that the object which appeared as an example at the start of this section, and which is not a member of the training set, should belong to class P. Notice that only a subset of the attributes may be encountered on a particular path from the root of the decision tree to a leaf; in this case, only the outlook attribute is tested before determining the class.

If the attributes are adequate, it is always possible to construct a decision tree that correctly classifies each object in the training set, and usually there are many such correct decision trees. The essence of induction is to move beyond the training set, i.e. to construct a decision tree that correctly classifies not only objects from the training set but other (unseen) objects as well. In order to do this, the decision tree must capture some meaningful relationship between an object's class and its values of the attributes. Given a choice between two decision trees, each of which is correct over the training set, it seems sensible to prefer the simpler one on the grounds that it is more likely to capture structure inherent in the problem. The simpler tree would therefore be expected to classify correctly more objects outside the training set. The decision tree of Figure 3, for instance, is also correct for the training set of Table 1, but its greater complexity makes it suspect as an 'explanation' of the training set.[2]

4. ID3

One approach to the induction task above would be to generate all possible decision trees that correctly classify the training set and to select the simplest of them. The

[2] The preference for simpler trees, presented here as a commonsense application of Occam's Razor, is also supported by analysis. Pearl (1978b) and Quinlan (1983a) have derived upper bounds on the expected error using different formalisms for generalizing from a set of known cases. For a training set of predetermined size, these bounds increase with the complexity of the induced generalization.

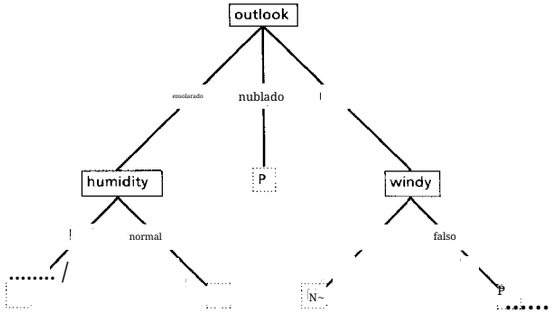


Figura 2. Uma árvore de decisão simples

a classe nomeada pela folha. Tomando a árvore de decisão da Figura 2, este processo conclui que o objeto que apareceu como exemplo no início desta seção, e que não é membro do conjunto de treinamento, deve pertencer à classe P. Observe que apenas um subconjunto dos atributos pode ser encontrado em um caminho particular da raiz da árvore de decisão até uma folha; neste caso, apenas o atributo Outlook é testado antes de determinar a classe.

Se os atributos forem adequados, é sempre possível construir uma árvore de decisão que classifique corretamente cada objeto no conjunto de treinamento, e geralmente existem muitas dessas árvores de decisão corretas. A essência da indução é i r além do conjunto de treinamento, ou seja, construir uma árvore de decisão que classifique corretamente não apenas os objetos do conjunto de treinamento, mas também outros objetos (invisíveis). Para fazer isso, a árvore de decisão deve capturar algum relacionamento significativo entre a classe de um objeto e os valores dos atributos. Dada a escolha entre duas árvores de decisão, cada uma delas correta em relação ao conjunto de treinamento, parece sensato preferir a mais simples, alegando que é mais provável que ela capture a estrutura inerente ao problema. Espera-se, portanto, que a árvore mais simples classifique corretamente mais objetos fora do conjunto de treinamento. A árvore de decisão da Figura 3, por exemplo, também é correta para o conjunto de treinamento da Tabela 1, mas sua maior complexidade a torna suspeita como uma "explicação" do conjunto de treinamento. 2

4.ID3

Uma abordagem para a tarefa de indução acima seria gerar todas as árvores de decisão possíveis que classifiquem corretamente o conjunto de treinamento e selecionar a mais simples delas. O

z A preferência por árvores mais simples, apresentada aqui como uma aplicação de bom senso da Navalha de Occam, também é apoiada pela análise. Pearl (1978b) e Quinlan (1983a) derivaram limites superiores para o erro esperado usando diferentes formalismos para generalizar a partir de um conjunto de casos conhecidos. Para um conjunto de treinamento de tamanho predeterminado, esses limites aumentam com a complexidade da generalização induzida.

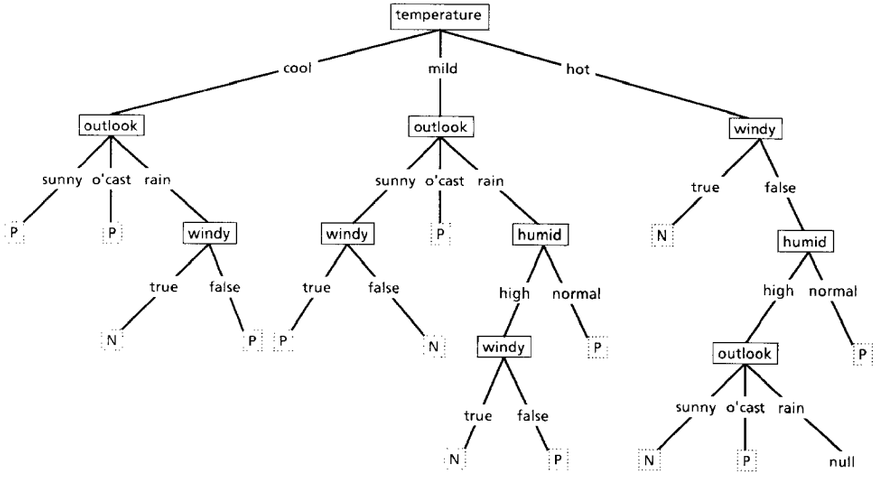


Figure 3. A complex decision tree.

number of such trees is finite but very large, so this approach would only be feasible for small induction tasks. ID3 was designed for the other end of the spectrum, where there are many attributes and the training set contains many objects, but where a reasonably good decision tree is required without much computation. It has generally been found to construct simple decision trees, but the approach it uses cannot guarantee that better trees have not been overlooked.

The basic structure of ID3 is iterative. A subset of the training set called the *window* is chosen at random and a decision tree formed from it; this tree correctly classifies all objects in the window. All other objects in the training set are then classified using the tree. If the tree gives the correct answer for all these objects then it is correct for the entire training set and the process terminates. If not, a selection of the incorrectly classified objects is added to the window and the process continues. In this way, correct decision trees have been found after only a few iterations for training sets of up to thirty thousand objects described in terms of up to 50 attributes. Empirical evidence suggests that a correct decision tree is usually found more quickly by this iterative method than by forming a tree directly from the entire training set. However, O'Keefe (1983) has noted that the iterative framework cannot be guaranteed to converge on a final tree unless the window can grow to include the entire training set. This potential limitation has not yet arisen in practice.

The crux of the problem is how to form a decision tree for an arbitrary collection C of objects. If C is empty or contains only objects of one class, the simplest decision tree is just a leaf labelled with the class. Otherwise, let T be any test on an object with possible outcomes $O_1, O_2, \ldots O_w$. Each object in C will give one of these outcomes for T, so T produces a partition $\{C_1, C_2, \ldots C_w\}$ of C with C_i containing those ob-

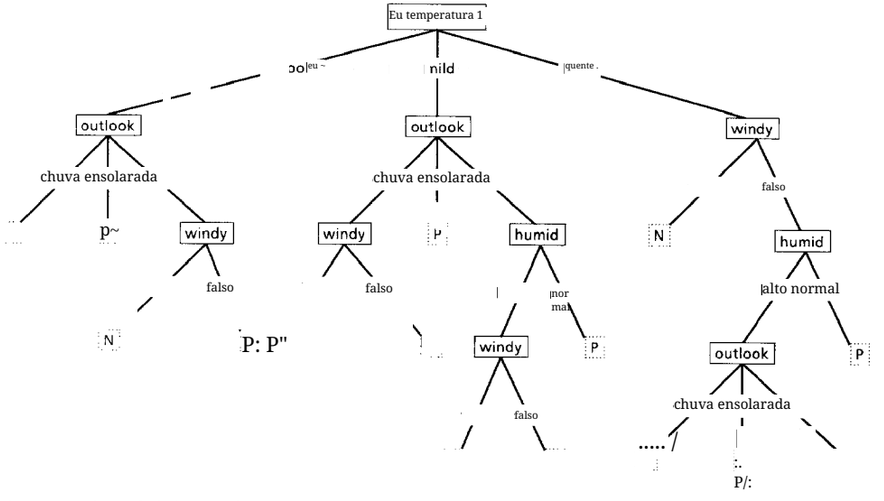


Figura 3. Uma árvore de decisão complexa.

O número dessas árvores é finito, mas muito grande, portanto esta abordagem só seria viável para pequenas tarefas de indução. O ID3 foi projetado para o outro extremo do espectro, onde existem muitos atributos e o conjunto de treinamento contém muitos objetos, mas onde uma árvore de decisão razoavelmente boa é necessária sem muita computação. Verificou-se geralmente que constrói árvores de decisão simples, mas a abordagem que utiliza não pode garantir que árvores melhores não tenham sido negligenciadas.

A estrutura básica do ID3 é iterativa. Um subconjunto do conjunto de treinamento denominado janela é escolhido aleatoriamente e uma árvore de decisão é formada a partir dele; esta árvore classifica corretamente todos os objetos na janela. Todos os outros objetos do conjunto de treinamento são então classificados usando a árvore. Se a árvore der a resposta correta para todos esses objetos, então ela estará correta para t odo o conjunto de treinamento e o processo será encerrado. Caso contrário, uma s eleção dos objetos classificados incorretamente será adicionada à janela e o processo continuará. Desta forma, foram encontradas árvores de decisão corretas após apenas algumas iterações para conjuntos de treinamento de até trinta mil objetos descritos em termos de até 50 atributos. Evidências empíricas sugerem que uma árvore de decisão correta geralmente é encontrada mais rapidamente por esse método iterativo do que pela formação de uma árvore diretamente a partir de todo o conjunto de treinamento. Contudo, O'Keefe (1983) observou que não é possível garantir que a estrutura iterativa convirja para uma árvore final, a menos que a janela possa crescer para incluir todo o conjunto de treinamento. Esta limitação potencial ainda não surgiu na prática.

O cerne do problema é como formar uma árvore de decisão para uma coleção arbitrária C de objetos. Se C estiver vazio ou contiver apenas objetos de uma classe, a árvore de decisão mais simples será apenas uma folha rotulada com a classe. Caso contrário, seja T qualquer teste em um objeto com resultados possíveis O1, Oz. . . . Ah. Cada objeto em C fornecerá um desses resultados para T, então T produz uma partição [C1, C2 . . . . Cw} de C com Ci contendo aqueles ob-

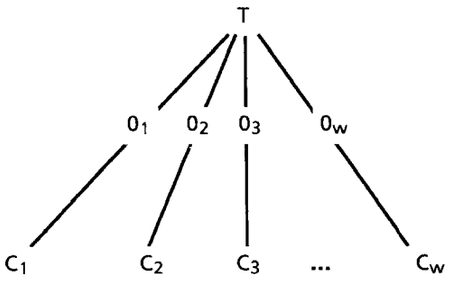


Figure 4. A tree structuring of the objects in C.

jects having outcome O_i. This is represented graphically by the tree form of Figure 4. If each subset C_i in this figure could be replaced by a decision tree for C_i, the result would be a decision tree for all of C. Moreover, so long as two or more C_i's are non-empty, each C_i is smaller than C. In the worst case, this divide-and-conquer strategy will yield single-object subsets that satisfy the one-class requirement for a leaf. Thus, provided that a test can always be found that gives a non-trivial partition of any set of objects, this procedure will always produce a decision tree that correctly classifies each object in C.

The choice of test is crucial if the decision tree is to be simple. For the moment, a test will be restricted to branching on the values of an attribute, so choosing a test comes down to selecting an attribute for the root of the tree. The first induction programs in the ID series used a seat-of-the-pants evaluation function that worked reasonably well. Following a suggestion of Peter Gacs, ID3 adopted an information-based method that depends on two assumptions. Let C contain p objects of class P and n of class N. The assumptions are:

(1) Any correct decision tree for C will classify objects in the same proportion as their representation in C. An arbitrary object will be determined to belong to class P with probability $p/(p+n)$ and to class N with probability $n/(p+n)$.
(2) When a decision tree is used to classify an object, it returns a class. A decision tree can thus be regarded as a source of a message 'P' or 'N', with the expected information needed to generate this message given by

$$I(p, n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

If attribute A with values $\{A_1, A_2, \ldots A_v\}$ is used for the root of the decision tree, it will partition C into $\{C_1, C_2, \ldots C_v\}$ where C_i contains those objects in C that have value A_i of A. Let C_i contain p_i objects of class P and n_i of class N. The expected

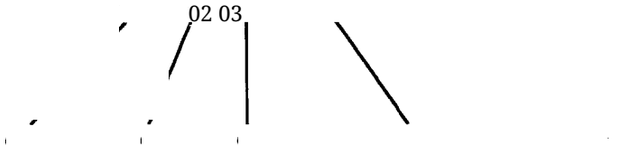


Figura 4. Estruturação em árvore dos objetos em C.

projetos tendo resultado Oi. Isso é representado graficamente pela forma de árvore da Figura 4. Se cada subconjunto Ci nesta figura pudesse ser substituído por uma árvore de decisão para Ci, o resultado seria uma árvore de decisão para todo C. Além disso, contanto que dois ou mais Ci não sejam vazios, cada Ci é menor que C. No pior caso, essa estratégia de dividir e conquistar produzirá subconjuntos de objeto único que satisfazem o requisito de uma classe para uma folha. Assim, desde que sempre seja possível encontrar um teste que forneça uma partição não trivial de qualquer conjunto de objetos, este procedimento sempre produzirá uma árvore de decisão que classifique corretamente cada objeto em C.

A escolha do teste é crucial para que a árvore de decisão seja simples. No momento, um teste estará restrito à ramificação dos valores de um atributo, portanto, a escolha de um teste se resume a selecionar um atributo para a raiz da árvore. Os primeiros programas de indução da série ID usaram uma função de avaliação imediata que funcionou razoavelmente bem. Seguindo uma sugestão de Peter Gacs, o ID3 adoptou um método baseado em informação que depende de dois pressupostos. Deixe C conter p objetos da classe P e n da classe N. As suposições são:

(1) Qualquer árvore de decisão correta para C classificará objetos na mesma proporção de sua representação em C. Um objeto arbitrário será determinado como pertencente à classe P com probabilidade $p/(p+n)$ e à classe N com probabilidade $n/(p+n)$. (2) Quando uma árvore de decisão é usada para classificar um objeto, ela retorna uma classe. Uma árvore de decisão pode assim ser considerada como uma fonte de uma mensagem 'P' ou 'N', com a informação esperada necessária para gerar esta mensagem dada por

$$I(p, n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

Se o atributo A tiver valores [AI, A2 .... Av] é usado para a raiz da árvore de decisão, -particionará C em [C1, Ca .... Cv] onde Ci contém aqueles objetos em C que possuem valor Ai de A. Deixe Ci conter objetos Pi da classe P e ni da classe N. O esperado

information required for the subtree for C_i is $I(p_i, n_i)$. The expected information required for the tree with A as root is then obtained as the weighted average

$$E(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I(p_i, n_i)$$

where the weight for the ith branch is the proportion of the objects in C that belong to C_i. The information gained by branching on A is therefore

$$gain(A) = I(p, n) - E(A)$$

A good rule of thumb would seem to be to choose that attribute to branch on which gains the most information.[3] ID3 examines all candidate attributes and chooses A to maximize gain(A), forms the tree as above, and then uses the same process recursively to form decision trees for the residual subsets $C_1, C_2, \ldots C_v$.

To illustrate the idea, let C be the set of objects in Table 1. Of the 14 objects, 9 are of class P and 5 are of class N, so the information required for classification is

$$I(p, n) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940 \text{ bits}$$

Now consider the outlook attribute with values {sunny, overcast, rain}. Five of the 14 objects in C have the first value (sunny), two of them from class P and three from class N, so

$$p_1 = 2 \quad n_1 = 3 \quad I(p_1, n_1) = 0.971$$

and similarly

$$p_2 = 4 \quad n_2 = 0 \quad I(p_2, n_2) = 0$$
$$p_3 = 3 \quad n_3 = 2 \quad I(p_3, n_3) = 0.971$$

The expected information requirement after testing this attribute is therefore

$$E(\text{outlook}) = \frac{5}{14} I(p_1, n_1) + \frac{4}{14} I(p_2, n_2) + \frac{5}{14} I(p_3, n_3)$$
$$= 0.694 \text{ bits}$$

[3] Since I(p,n) is constant for all attributes, maximizing the gain is equivalent to minimizing E(A), which is the mutual information of the attribute A and the class. Pearl (1978a) contains an excellent account of the rationale of information-based heuristics.

a informação necessária para a subárvore de Ci é I(pi, ni). A informação esperada necessária para a árvore com A como raiz é então obtida como a média ponderada

$$E(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I(p_i, n_i)$$

onde o peso do i-ésimo ramo é a proporção dos objetos em C que pertencem a Ci. A informação obtida pela ramificação em A é, portanto,

$$gain(A) = I(p, n) - E(A)$$

Uma boa regra parece ser escolher aquele atributo no ramo que obtém mais informações. 3 ID3 examina todos os atributos candidatos e escolhe A para maximizar o ganho(A), forma a árvore como acima e então usa o mesmo processo recursivamente para formar árvores de decisão para os subconjuntos residuais C1, CE. . . . Cv.

Para ilustrar a ideia, seja C o conjunto de objetos da Tabela 1. Dos 14 objetos, 9 são da classe P e 5 são da classe N, portanto a informação necessária para classificação é

$$I(p, n) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940 \text{ bits}$$

Agora considere o atributo Outlook com valores [ensolarado, nublado, chuva}. Cinco dos 14 objetos em C têm o primeiro valor (ensolarado), dois deles da classe P e três da classe N, então

$$p_1 = 2 \quad n_1 = 3 \quad I(p_1, n_1) = 0.971$$

e da mesma forma

$$p_2 = 4 \quad n_2 = 0 \quad I(p_2, n_2) = 0$$
$$p_3 = 3 \quad n_3 = 2 \quad I(p_3, n_3) = 0.971$$

O requisito de informação esperado após testar este atributo é, portanto,

$$E(\text{outlook}) = \frac{5}{14} I(p_1, n_1) + \frac{4}{14} I(p_2, n_2) + \frac{5}{14} I(p_3, n_3)$$
$$= 0.694 \text{ bits}$$

3 Como I(p,n) é constante para todos os atributos, maximizar o ganho equivale a minimizar E(A), que é a informação mútua do atributo A e da classe. Pearl (1978a) contém um excelente relato da lógica da heurística baseada em informação.

The gain of this attribute is then

$$\text{gain(outlook)} = 0.940 - \text{E(outlook)} = 0.246 \text{ bits}$$

Similar analysis gives

$$\text{gain(temperature)} = 0.029 \text{ bits}$$
$$\text{gain(humidity)} = 0.151 \text{ bits}$$
$$\text{gain(windy)} = 0.048 \text{ bits}$$

so the tree-forming method used in ID3 would choose outlook as the attribute for the root of the decision tree. The objects would then be divided into subsets according to their values of the outlook attribute and a decision tree for each subset would be induced in a similar fashion. In fact, Figure 2 shows the actual decision tree generated by ID3 from this training set.

A special case arises if C contains no objects with some particular value A_j of A, giving an empty C_j. ID3 labels such a leaf as 'null' so that it fails to classify any object arriving at that leaf. A better solution would generalize from the set C from which C_j came, and assign this leaf the more frequent class in C.

The worth of ID3's attribute-selecting heuristic can be assessed by the simplicity of the resulting decision trees, or, more to the point, by how well those trees express real relationships between class and attributes as demonstrated by the accuracy with which they classify objects other than those in the training set (their *predictive accuracy*). A straightforward method of assessing this predictive accuracy is to use only part of the given set of objects as a training set, and to check the resulting decision tree on the remainder.

Several experiments of this kind have been carried out. In one domain, 1.4 million chess positions described in terms of 49 binary-valued attributes gave rise to 715 distinct objects divided 65%:35% between the classes. This domain is relatively complex since a correct decision tree for all 715 objects contains about 150 nodes. When training sets containing 20% of these 715 objects were chosen at random, they produced decision trees that correctly classified over 84% of the unseen objects. In another version of the same domain, 39 attributes gave 551 distinct objects with a correct decision tree of similar size; training sets of 20% of these 551 objects gave decision trees of almost identical accuracy. In a simpler domain (1,987 objects with a correct decision tree of 48 nodes), randomly-selected training sets containing 20% of the objects gave decision trees that correctly classified 98% of the unseen objects. In all three cases, it is clear that the decision trees reflect useful (as opposed to random) relationships present in the data.

This discussion of ID3 is rounded off by looking at the computational requirements of the procedure. At each non-leaf node of the decision tree, the gain of each untested attribute A must be determined. This gain in turn depends on the values p_i

O ganho deste atributo é então

$$\text{gain(outlook)} = 0.940 - \text{E(outlook)} = 0.246 \text{ bits}$$

Análise semelhante dá

ganho (temperatura) = 0 ganho de
0,029 bits (umidade) = 0 ganho de
0,151 bits (vento) = 0 0,048 bits

portanto, o método de formação de árvore usado no ID3 escolheria o Outlook como o atributo para a raiz da árvore de decisão. Os objetos seriam então divididos em s ubconjuntos de acordo com seus valores do atributo de perspectiva e uma árvore de decisão para cada subconjunto seria induzida de maneira semelhante. Na verdade, a Figura 2 mostra a árvore de decisão real gerada pelo ID3 a partir deste conjunto de treinamento.

Um caso especial surge se C não contém objetos com algum valor particular Aj de A, dando um Cj vazio. O ID3 rotula essa folha como 'nula' para que não consiga classificar qualquer objeto que chegue a essa folha. Uma solução melhor seria generalizar a partir do conjunto C do qual veio Cj e atribuir a esta folha a classe mais frequente em C.

O valor da heurística de seleção de atributos do ID3 pode ser avaliado pela simplicidade das árvores de decisão resultantes ou, mais precisamente, pela forma como essas árvores expressam relações reais entre classe e atributos, conforme demonstrado pela precisão com que classificam objetos que não sejam aqueles no conjunto de treinamento (sua precisão preditiva). Um método simples de avaliar essa precisão preditiva é usar apenas parte de um determinado conjunto de objetos como conjunto de treinamento e verificar a árvore de decisão resultante no restante.

Várias experiências deste tipo foram realizadas. Num domínio, 1,4 milhões de posições de xadrez descritas em termos de 49 atributos de valor binário deram origem a 715 objetos distintos divididos 65%:35% entre as classes. Este domínio é relativamente complexo, pois uma árvore de decisão correta para todos os 715 objetos contém cerca de 150 nós. Quando conjuntos de treinamento contendo 20% desses 715 objetos foram escolhidos aleatoriamente, eles produziram árvores de decisão que classificaram corretamente mais de 84% dos objetos invisíveis. Em outra versão do mesmo domínio, 39 atributos deram 551 objetos distintos com uma árvore de decisão correta de t amanho semelhante; conjuntos de treinamento de 20% desses 551 objetos forneceram árvores de decisão com precisão quase idêntica. Em um domínio mais simples (1.987 objetos com uma árvore de decisão correta de 48 nós), conjuntos de treinamento selecionados aleatoriamente contendo 20°7o dos objetos forneceram árvores de decisão que classificaram corretamente 98% dos objetos invisíveis. Nos três casos, é claro que as árvores de decisão reflectem relações úteis (em oposição a aleatórias) presentes nos dados.

Esta discussão do ID3 é completada pela observação dos requisitos computacionais do procedimento. Em cada nó não-folha da árvore de decisão, o ganho de cada atributo A não testado deve ser determinado. Este ganho, por sua vez, depende dos valores pi

and n_i for each value A_i of A, so every object in C must be examined to determine its class and its value of A. Consequently, the computational complexity of the procedure at each such node is $O(|C| \cdot |A|)$, where $|A|$ is the number of attributes above. ID3's total computational requirement per iteration is thus proportional to the product of the size of the training set, the number of attributes and the number of non-leaf nodes in the decision tree. The same relationship appears to extend to the entire induction process, even when several iterations are performed. No exponential growth in time or space has been observed as the dimensions of the induction task increase, so the technique can be applied to large tasks.

5. Noise

So far, the information supplied in the training set has been assumed to be entirely accurate. Sadly, induction tasks based on real-world data are unlikely to find this assumption to be tenable. The description of objects may include attributes based on measurements or subjective judgements, both of which may give rise to errors in the values of attributes. Some of the objects in the training set may even have been misclassified. To illustrate the idea, consider the task of developing a classification rule for medical diagnosis from a collection of patient histories. An attribute might test for the presence of some substance in the blood and will almost inevitably give false positive or negative readings some of the time. Another attribute might assess the patient's build as slight, medium, or heavy, and different assessors may apply different criteria. Finally, the collection of case histories will probably include some patients for whom an incorrect diagnosis was made, with consequent errors in the class information provided in the training set.

What problems might errors of these kinds pose for the tree-building procedure described earlier? Consider again the small training set in Table 1, and suppose now that attribute outlook of object 1 is incorrectly recorded as overcast. Objects 1 and 3 will then have identical descriptions but belong to different classes, so the attributes become inadequate for this training set. The attributes will also become inadequate if attribute windy of object 4 is corrupted to true, because that object will then conflict with object 14. Finally, the initial training set can be accounted for by the simple decision tree of Figure 2 containing 8 nodes. Suppose that the class of object 3 were corrupted to N. A correct decision tree for this corrupted training set would now have to explain the apparent special case of object 3. The smallest such tree contains twelve nodes, half again as complex as the 'real' tree. These illustrations highlight two problems: errors in the training set may cause the attributes to become inadequate, or may lead to decision trees of spurious complexity.

Non-systematic errors of this kind in either the values of attributes or class information are usually referred to as *noise*. Two modifications are required if the tree-building algorithm is to be able to operate with a noise-affected training set.

e ni para cada valor Ai de A, então todo objeto em C deve ser examinado para determinar sua classe e seu valor de A. Conseqüentemente, a complexidade computacional do procedimento em cada um desses nós é O (I C I . IAI), onde IAI é o número de atributos acima. O requisito computacional total do ID3 por iteração é, portanto, proporcional ao produto do tamanho do conjunto de treinamento, ao nú mero de atributos e ao número de nós não-folha na árvore de decisão. A mesma rel ação parece estender-se a todo o processo de indução, mesmo quando são realizadas diversas iterações. Nenhum crescimento exponencial no tempo ou no espaço foi observado à medida que as dimensões da tarefa de indução aumentam, de modo que a técnica pode ser aplicada a tarefas grandes.

5. Ruído

Até agora, as informações fornecidas no conjunto de treinamento foram consideradas totalmente precisas. Infelizmente, é improvável que tarefas de indução baseadas em dados do mundo real considerem esta suposição sustentável. A descrição de objetos pode incluir atributos baseados em medições ou julgamentos subjetivos, os quais podem dar origem a erros nos valores dos atributos. Alguns dos objetos do conjunto de treinamento podem até ter sido classificados incorretamente. Para ilustrar a ideia, consideremos a tarefa de desenvolver uma regra de classificação para diagnóstico médico a partir de uma coleção de históricos de pacientes. Um atributo pode testar a presença de alguma substância no sangue e quase inevitavelmente fornecerá leituras falsas positivas ou negativas em algum momento. Outro atributo pode avaliar a constituição do paciente como leve, média ou pesada, e diferentes avaliadores podem aplicar critérios diferentes. Finalmente, a recolha de histórias de casos provavelmente incluirá alguns pacientes para os quais foi feito um diagnóstico incorrecto, com consequentes erros nas informações de classe fornecidas no conjunto de treino.

Que problemas erros desse tipo podem representar para o procedimento de construção de árvore descrito anteriormente? Considere novamente o pequeno conjunto de treinamento na Tabela 1 e suponha agora que a perspectiva do atributo do objeto 1 seja registrada incorretamente como nublada. Os objetos 1 e 3 terão então descrições idênticas, mas pertencerão a classes diferentes, portanto os atributos tornam-se inadequados para este conjunto de treinamento. Os atributos também se tornarão inadequados se o atributo windy do objeto 4 for corrompido para verdadeiro, porque esse objeto entrará em conflito com o objeto 14. Finalmente, o conjunto de treinamento inicial pode ser explicado pela árvore de decisão simples da Figura 2 contendo 8 nós. Suponha que a classe do objeto 3 foi corrompida para N. Uma árvore de decisão correta para esse conjunto de treinamento corrompido teria agora que explicar o aparente caso especial do objeto 3. A menor dessas árvores contém doze nós, novamente com metade da complexidade da árvore 'real'. Estas ilustrações destacam dois problemas: erros no conjunto de treinamento podem fazer com que os atributos se tornem inadequados ou podem levar a árvores de decisão de complexidade espúria.

Erros não sistemáticos desse tipo, tanto nos valores dos atributos quanto nas informações de classe, são geralmente chamados de ruído. Duas modificações são necessárias para que o algoritmo de construção de árvore seja capaz de operar com um conjunto de treinamento afetado por ruído.

(1) The algorithm must be able to work with inadequate attributes, because noise can cause even the most comprehensive set of attributes to appear inadequate.
(2) The algorithm must be able to decide that testing further attributes will not improve the predictive accuracy of the decision tree. In the last example above, it should refrain from increasing the complexity of the decision tree to accommodate a single noise-generated special case.

We start with the second requirement of deciding when an attribute is really relevant to classification. Let C be a collection of objects containing representatives of both classes, and let A be an attribute with random values that produces subsets $\{C_1, C_2, \ldots C_v\}$. Unless the proportion of class P objects in each of the C_i is exactly the same as the proportion of class P objects in C itself, branching on attribute A will give an apparent information gain. It will therefore appear that testing attribute A is a sensible step, even though the values of A are random and so cannot help to classify the objects in C.

One solution to this dilemma might be to require that the information gain of any tested attribute exceeds some absolute or percentage threshold. Experiments with this approach suggest that a threshold large enough to screen out irrelevant attributes also excludes attributes that are relevant, and the performance of the tree-building procedure is degraded in the noise-free case.

An alternative method based on the chi-square test for stochastic independence has been found to be more useful. In the previous notation, suppose attribute A produces subsets $\{C_1, C_2, \ldots C_v\}$ of C, where C_i contains p_i and n_i objects of class P and N, respectively. If the value of A is irrelevant to the class of an object in C, the expected value p'_i of p_i should be

$$p'_i = p \cdot \frac{p_i + n_i}{p + n}$$

If n'_i is the corresponding expected value of n_i, the statistic

$$\sum_{i=1}^{v} \frac{(p_i - p'_i)^2}{p'_i} + \frac{(n_i - n'_i)^2}{n'_i}$$

is approximately chi-square with v-1 degrees of freedom. Provided that none of the values p'_i or n'_i are very small, this statistic can be used to determine the confidence with which one can reject the hypothesis that A is independent of the class of objects in C (Hogg and Craig, 1970). The tree-building procedure can then be modified to prevent testing any attribute whose irrelevance cannot be rejected with a very high (e.g. 99%) confidence level. This has been found effective in preventing over-

(1) O algoritmo deve ser capaz de trabalhar com atributos inadequados, porque o ruído pode fazer com que até mesmo o conjunto mais abrangente de atributos pareça inadequado. "(2) O algoritmo deve ser capaz de decidir que testar atributos adicionais não melhorará a precisão preditiva da árvore de decisão. No último exemplo acima, ele deve evitar aumentar a complexidade da árvore de decisão para acomodar um único caso especial gerado por ruído.

Começamos com o segundo requisito de decidir quando um atributo é realmente relevante para a classificação. Seja C uma coleção de objetos contendo representantes de ambas as classes, e seja A um atributo com valores aleatórios que produz subconjuntos { C1, C2 . . . . Cv 1. A menos que a proporção de objetos da classe P em cada um dos Ci seja exatamente a mesma que a proporção de objetos da classe P no próprio C, a ramificação no atributo A proporcionará um aparente ganho de informação. Parecerá, portanto, que testar o atributo A é um passo sensato, mesmo que os valores de A sejam aleatórios e, portanto, não possam ajudar a classificar os objetos em C.

Uma solução para este dilema poderia ser exigir que o ganho de informação de qualquer atributo testado exceda algum limite absoluto ou percentual. Experimentos com esta abordagem sugerem que um limite grande o suficiente para excluir atributos irrelevantes também exclui atributos que são relevantes, e o desempenho do procedimento de construção de árvore é degradado no caso sem ruído.

Um método alternativo baseado no teste qui-quadrado para independência estocástica revelou-se mais útil. Na notação anterior, suponha que o atributo A produza os subconjuntos 1C1, C2. . . . Cv} de C, onde Ci contém objetos pi e ni das classes P e N, respectivamente. Se o valor de A for irrelevante para a classe de um objeto em C, o valor esperado~p 'i de pi deve ser

$$p'_i = p \cdot \frac{p_i + n_i}{p + n}$$

Se n 'i for o valor esperado correspondente de ni, a estatística

$$\sum_{i=1}^{v} \frac{(p_i - p'_i)^2}{p'_i} + \frac{(n_i - n'_i)^2}{n'_i}$$

é aproximadamente qui-quadrado com graus de liberdade v-1. Desde que nenhum dos valores p'i ou n'i seja muito pequeno, esta estatística pode ser usada para determinar a confiança com a qual se pode rejeitar a hipótese de que A é independente da classe de objetos em C (Hogg e Craig, 1970). O procedimento de construção de árvore pode então ser modificado para evitar testar qualquer atributo cuja irrelevância não possa ser rejeitada com um nível de confiança muito alto (por exemplo, 99%). Isto foi c onsiderado eficaz na prevenção de excesso de

complex trees that attempt to 'fit the noise' without affecting performance of the procedure in the noise-free case.[4]

Turning now to the first requirement, we see that the following situation can arise: a collection of C objects may contain representatives of both classes, yet further testing of C may be ruled out, either because the attributes are inadequate and unable to distinguish among the objects in C, or because each attribute has been judged to be irrelevant to the class of objects in C. In this situation it is necessary to produce a leaf labelled with class information, but the objects in C are not all of the same class.

Two possibilities suggest themselves. The notion of class could be generalized to allow the value $p/(p+n)$ in the interval (0,1), a class of 0.8 (say) being interpreted as 'belonging to class P with probability 0.8'. An alternative approach would be to opt for the more numerous class, i.e. to assign the leaf to class P if $p > n$, to class N if $p < n$, and to either if $p = n$. The first approach minimizes the sum of the squares of the error over objects in C, while the second minimizes the sum of the absolute errors over objects in C. If the aim is to minimize expected error, the second approach might be anticipated to be superior, and indeed this has been found to be the case.

Several studies have been carried out to see how this modified procedure holds up under varying levels of noise (Quinlan 1983b, 1985a). One such study is outlined here based on the earlier-mentioned task with 551 objects and 39 binary-valued attributes. In each experiment, the whole set of objects was artificially corrupted as described below and used as a training set to produce a decision tree. Each object was then corrupted anew, classified by this tree and the error rate determined. This process was repeated twenty times to give more reliable averages.

In this study, values were corrupted as follows. A noise level of n percent applied to a value meant that, with probability n percent, the true value was replaced by a value chosen at random from among the values that could have appeared.[5] Table 2 shows the results when noise levels varying from 5% to 100% were applied to the values of the most noise-sensitive attribute, to the values of all attributes simultaneously, and to the class information. This table demonstrates the quite different forms of degradation observed. Destroying class information produces a linear increase in error so that, when all class information is noise, the resulting decision tree classifies objects entirely randomly. Noise in a single attribute does not have a dramatic effect. Noise in all attributes together, however, leads to a relatively rapid increase in error which reaches a peak and declines. The peak is somewhat inter-

[4] ASSISTANT uses an information-based measure to perform much the same function, but no comparative results are available to date.

[5] It might seem that the value should be replaced by an incorrect value. Consider, however, the case of a two-valued attribute corrupted with 100% noise. If the value of each object were replaced by the (only) incorrect value, the initial attribute will have been merely inverted with no loss of information.

árvores complexas que tentam 'ajustar-se ao ruído' sem afetar o desempenho do procedimento no caso sem ruído. 4

Voltando-nos agora para o primeiro requisito, vemos que a seguinte situação pode surgir: uma coleção de objetos C pode conter representantes de ambas as classes, mas testes adicionais de C podem ser descartados, ou porque os atributos são inadequados e incapazes de distinguir entre os objetos em C, ou porque cada atributo foi considerado irrelevante para a classe de objetos em C. Nesta situação, é necessário produzir uma folha rotulada co m informações de classe. rm ation , mas os objetos em C não são todos da mesma classe.

Duas possibilidades se apresentam. A noção de classe poderia ser generalizada para permitir o valor $p / (p + n)$ no intervalo (0,1), uma classe de 0,8 (digamos) sendo i nterpretada como 'pertencente à classe P com probabilidade de 0,8'. Uma abordagem alternativa seria optar pela classe mais numerosa, ou seja, atribuir a folha à classe P se $p > n$, à classe N se $p < n$, e a qualquer uma delas se $p = n$. A primeira abordagem minimiza a soma dos quadrados do erro sobre os objetos em C, enquanto a segunda minimiza a soma dos erros absolutos sobre os objetos em C. Se o objetivo é minimizar o erro esperado, a segunda abordagem pode ser antecipada como superior, e de fato descobriu-se que é esse o caso.

Vários estudos foram realizados para ver como esse procedimento modificado se comporta sob vários níveis de ruído (Quinlan 1983b, 1985a). Um desses estudos é descrito aqui com base na tarefa mencionada anteriormente com 551 objetos e 39 atributos de valor binário. Em cada experimento, todo o conjunto de objetos foi corrompido artificialmente conforme descrito abaixo e usado como um conjunto de treinamento para produzir uma árvore de decisão. Cada objeto foi então corrompido novamente, classificado por esta árvore e a taxa de erro determinada. Este processo foi repetido vinte vezes para fornecer médias mais confiáveis.

Neste estudo, os valores foram corrompidos da seguinte forma. Um nível de ruído de n por cento aplicado a um valor significava que, com probabilidade n por cento, o valor verdadeiro era substituído por um valor escolhido aleatoriamente entre os valores que poderiam ter aparecido. 5 A Tabela 2 mostra os resultados quando níveis de ruído variando de 5% a 100% foram aplicados aos valores do atributo mais sensível ao ruído, aos valores de todos os atributos simultaneamente e às informações da classe. Esta tabela demonstra as diferentes formas de degradação observadas. Destruir informações de classe produz um aumento linear no erro de modo que, quando todas as informações de classe são ruído, a árvore de decisão resultante classifica objetos de forma inteiramente aleatória. O ruído em um único atributo não tem um efeito dramático. O ruído em todos os atributos juntos, entretanto, leva a um aumento relativamente rápido no erro que atinge um pico e diminui. O pico é um tanto inter-

4 O ASSISTANT utiliza uma medida baseada em informações para desempenhar praticamente a mesma função, mas nenhum resultado comparativo está disponível até o momento. 5 Pode parecer que o valor deveria ser substituído por um valor incorreto. Considere, entretanto, o caso de um atributo de dois valores corrompido com 100% de ruído. Se o valor de cada objeto for substituído pelo (único) valor incorreto, o atributo inicial terá sido meramente invertido sem perda de informação.

Table 2. Error rates produced by noise in a single attribute, all attributes, and class information

Noise level	Single attribute	All attributes	Class information
5%	1.3%	11.9%	2.6%
10%	2.5%	18.9%	5.5%
15%	3.3%	24.6%	8.3%
20%	4.6%	27.8%	9.9%
30%	6.1%	29.5%	14.8%
40%	7.6%	30.3%	18.1%
50%	8.8%	29.2%	21.8%
60%	9.4%	27.5%	26.4%
70%	9.9%	25.9%	27.2%
80%	10.4%	26.0%	29.5%
90%	10.8%	25.6%	34.1%
100%	10.8%	25.9%	49.6%

esting, and can be explained as follows. Let C be a collection of objects containing p from class P and n from class N, respectively. At noise levels around 50%, the algorithm for constructing decision trees may still find relevant attributes to branch on, even though the performance of this tree on unseen but equally noisy objects will be essentially random. Suppose the tree for C classifies objects as class P with probability $p/(n+p)$. The expected error if objects with a similar class distribution to those in C were classified by this tree is given by

$$\frac{p}{p+n} \cdot \left(1 - \frac{p}{p+n}\right) + \frac{n}{p+n} \cdot \left(1 - \frac{n}{p+n}\right) = \frac{2pn}{(p+n)^2}$$

At very high levels of noise, however, the algorithm will find all attributes irrelevant and classify everything as the more frequent class; assume without loss of generality that this class is P. The expected error in this case is

$$\frac{p}{p+n} \cdot 0 + \frac{n}{p+n} \cdot 1 = \frac{n}{p+n}$$

which is less than the above expression since we have assumed that p is greater than n. The decline in error is thus a consequence of the chi-square cutoff coming into play as noise becomes more intense.

The table brings out the point that low levels of noise do not cause the tree-building machinery to fall over a cliff. For this task, a 5% noise level in a single attribute produces a degradation in performance of less than 2%; a 5% noise level in all attributes together produces a 12% degradation in classification performance; while a similar

Tabela 2. Taxas de erro produzidas por ruído em um único atributo, todos os atributos e informações de classe

Barulho nível	Solteiro atributo	atributos	Aula Informação
		11,9%	
		18,9%	
		24,6%	
		27,80/0	
		29,5%	14,8%
		30,3%	18,1%
		29,2%	21,8%
		27,5%	
		25,9%	27,2%
	10,4%	26,0%	29,5%
	10,8%	25,6%	34,1%
	10,8%	25,9%	49,6%

esting, e pode ser explicado da seguinte forma. Seja C uma coleção de objetos contendo p da classe P e n da classe N, respectivamente. Em níveis de ruído em torno de 50%, o algoritmo para construir árvores de decisão ainda pode encontrar atributos relevantes para ramificar, mesmo que o desempenho desta árvore em objetos invisíveis, mas igualmente barulhentos, seja essencialmente aleatório. Suponha que a árvore para C classifique objetos como classe P com probabilidade $p/(n + p)$. O erro esperado se objetos com distribuição de classes semelhante àquelas em C fossem classificados por esta árvore é dado por

$$\frac{p}{p+n} \cdot \left(1 - \frac{p}{p+n}\right) + \frac{n}{p+n} \cdot \left(1 - \frac{n}{p+n}\right) = \frac{2pn}{(p+n)^2}$$

Em níveis muito elevados de ruído, entretanto, o algoritmo considerará todos os atributos irrelevantes e classificará tudo como a classe mais frequente; assuma sem perda de generalidade que esta classe é P. O erro esperado neste caso é

$$\frac{p}{p+n} \cdot 0 + \frac{n}{p+n} \cdot 1 = \frac{n}{p+n}$$

que é menor que a expressão acima, pois assumimos que p é maior que n. O declínio no erro é, portanto, uma consequência do corte do qui-quadrado entrar em ação à medida que o ruído se torna mais intenso.

A tabela salienta que baixos níveis de ruído não fazem com que as máquinas de construção de árvores caiam de um penhasco. Para esta tarefa, um nível de ruído de 5°7o em um único atributo produz uma degradação no desempenho inferior a 2%; um nível de ruído de 5 a 70 em todos os atributos juntos produz uma degradação de 12% no desempenho da classificação; enquanto um semelhante

noise level in class information results in a 3% degradation. Comparable figures have been obtained for other induction tasks.

One interesting point emerged from other experiments in which a correct decision tree formed from an uncorrupted training set was used to classify objects whose descriptions were corrupted. This scenario corresponds to forming a classification rule under controlled and sanitized laboratory conditions, then using it to classify objects in the field. For higher noise levels, the performance of the correct decision tree on corrupted data was found to be inferior to that of an imperfect decision tree formed from data corrupted to a similar level! (This phenomenon has an explanation similar to that given above for the peak in Table 2.) The moral seems to be that it is counter-productive to eliminate noise from the attribute information in the training set if these same attributes will be subject to high noise levels when the induced decision tree is put to use.

6. Unknown attribute values

The previous section examined modifications to the tree-building process that enabled it to deal with noisy or corrupted values. This section is concerned with an allied problem that also arises in practice: unknown attribute values. To continue the previous medical diagnosis example, what should be done when the patient case histories that are to form the training set are incomplete?

One way around the problem attempts to fill in an unknown value by utilizing information provided by context. Using the previous notation, let us suppose that a collection C of objects contains one whose value of attribute A is unknown. ASSISTANT (Kononenko *et al*, 1984) uses a Bayesian formalism to determine the probability that the object has value A_i of A by examining the distribution of values of A in C as a function of their class. Suppose that the object in question belongs to class P. The probability that the object has value A_i for attribute A can be expressed as

$$\text{prob}(A = A_i \mid \text{class} = P) = \frac{\text{prob}(A = A_i \ \& \ \text{class} = P)}{\text{prob}(\text{class} = P)} = \frac{p_i}{p}$$

where the calculation of p_i and p is restricted to those members of C whose value of A is known. Having determined the probability distribution of the unknown value over the possible values of A, this method could either choose the most likely value or divide the object into fractional objects, each with one possible value of A, weighted according to the probabilities above.

Alen Shapiro (private communication) has suggested using a decision-tree approach to determine the unknown values of an attribute. Let C′ be the subset of C consisting of those objects whose value of attribute A is defined. In C′, the original

o nível de ruído nas informações da classe resulta em uma degradação de 3°70. Valores comparáveis foram obtidos para outras tarefas de indução.

Um ponto interessante surgiu de outros experimentos nos quais uma árvore de decisão correta formada a partir de um conjunto de treinamento não corrompido foi usada para classificar objetos cujas descrições estavam corrompidas. Este cenário corresponde a formar uma regra de classificação em condições laboratoriais controladas e higienizadas e utilizá-la para classificar objetos em campo. Para níveis de ruído mais elevados, o desempenho da árvore de decisão correta em dados corrompidos foi considerado inferior ao de uma árvore de decisão imperfeita formada a partir de dados corrompidos em um nível semelhante! (Este fenômeno tem uma explicação semelhante àquela dada acima para o pico na Tabela 2.) A moral parece ser que é contraproducente eliminar o ruído da informação do atributo no conjunto de treinamento se esses mesmos atributos estiverem sujeitos a altos níveis de ruído quando a árvore de decisão induzida for colocada em uso.

6. Valores de atributos desconhecidos

A seção anterior examinou modificações no processo de construção de árvore que lhe permitiram lidar com valores ruidosos ou corrompidos. Esta seção trata de um problema aliado que também surge na prática: valores de atributos desconhecidos. Para continuar o exemplo de diagnóstico médico anterior, o que deve ser feito quando os históricos de casos dos pacientes que formarão o conjunto de treinamento estão incompletos?

Uma maneira de contornar o problema tenta preencher um valor desconhecido utilizando informações fornecidas pelo contexto. Usando a notação anterior, suponhamos que uma coleção C de objetos contém um cujo valor do atributo A é desconhecido. ASSIS T A N T (Kononenko et al, 1984) utiliza um formalismo bayesiano para determinar a probabilidade de o objeto ter valor Ai de A examinando a distribuição de valores de A em C em função de sua classe. Suponha que o objeto em questão pertença à classe P. A probabilidade de o objeto ter valor Ai para o atributo A pode ser expressa como

$$\text{prob}(A = A_i \mid \text{class} = P) = \frac{\text{prob}(A = A_i \ \& \ \text{class} = P)}{\text{prob}(\text{class} = P)} = \frac{p_i}{p}$$

onde o cálculo de Pi e p é restrito aos membros de C cujo valor de A é conhecido. Tendo determinado a distribuição de probabilidade do valor desconhecido sobre os valores possíveis de A, este método poderia escolher o valor mais provável ou dividir o objeto em objetos fracionários, cada um com um valor possível de A, ponderado de acordo com as probabilidades acima.

Alert Shapiro (comunicação privada) sugeriu o uso de uma abordagem de árvore de decisão para determinar os valores desconhecidos de um atributo. Seja C ' o s ubconjunto de C que consiste naqueles objetos cujo valor do atributo A é definido.

Table 3. Proportion of times that an unknown attribute value is replaced by an incorrect value

Replacement method	Attribute		
	1	2	3
Bayesian	28%	27%	38%
Decision tree	19%	22%	19%
Most common value	28%	27%	40%

class (P or N) is regarded as another attribute while the value of attribute A becomes the 'class' to be determined. That is, C′ is used to construct a decision tree for determining the value of attribute A from the other attributes and the class. When constructed, this decision tree can be used to 'classify' each object in C − C′ and the result assigned as the unknown value of A.

Although these methods for determining unknown attribute values look good on paper, they give unconvincing results even when only a single value of one attribute is unknown; as might be expected, their performance is much worse when several values of several attributes are unknown. Consider again the 551-object 39-attribute task. We may ask how well the methods perform when asked to fill in a single unknown attribute value. Table 3 shows, for each of the three most important attributes, the proportion of times each method fails to replace an unknown value by its correct value. For comparison, the table also shows the same figure for the simple strategy: always replace an unknown value of an attribute with its most common value. The Bayesian method gives results that are scarcely better than those given by the simple strategy and, while the decision-tree method uses more context and is thereby more accurate, it still gives disappointing results.

Rather than trying to guess unknown attribute values, we could treat 'unknown' as a new possible value for each attribute and deal with it in the same way as other values. This can lead to an anomalous situation, as shown by the following example. Suppose A is an attribute with values $\{A_1, A_2\}$ and let C be a collection of objects such that

$$p_1 = 2 \quad p_2 = 2$$
$$n_1 = 2 \quad n_2 = 2$$

giving a value of 1 bit for E(A). Now let A′ be an identical attribute except that one of the objects with value A_1 of A has an unknown value of A′. A′ has the values $\{A'_1, A'_2, A'_3 = \text{unknown}\}$, so the corresponding values might be

$$p'_1 = 1 \quad p'_2 = 2 \quad p'_3 = 1$$
$$n'_1 = 2 \quad n'_2 = 2 \quad n'_3 = 0$$

Tabela 3. Proporção de vezes que um valor de atributo desconhecido é substituído por um valor incorreto

Substituição método	Atributo		
Bayesiano			38°7o
Árvore de decisão			
Valor mais comum	28°7o	27°70	4007o

a classe (P ou N) é considerada outro atributo, enquanto o valor do atributo A torna-se a 'classe' a ser determinada. Ou seja, C' é usado para construir uma árvore de decisão para determinar o valor do atributo A a partir dos demais atributos e da classe. Quando construída, esta árvore de decisão pode ser usada para 'classificar' cada objeto em C - C ' e o resultado atribuído como o valor desconhecido de A.

Embora estes métodos para determinar valores de atributos desconhecidos pareçam bons no papel, eles fornecem resultados pouco convincentes mesmo quando apenas um único valor de um atributo é desconhecido; como seria de esperar, o seu desempenho é muito pior quando vários valores de vários atributos são desconhecidos. Considere novamente a tarefa de 551 objetos e 39 atributos. Podemos perguntar quão bem os métodos funcionam quando solicitados a preencher um único valor de atributo desconhecido. A Tabela 3 mostra, para cada um dos três atributos mais importantes, a proporção de vezes que cada método não consegue substituir um valor desconhecido pelo seu valor correto. Para efeito de comparação, a tabela também mostra a mesma figura para a estratégia simples: sempre substitua um valor desconhecido de um atributo pelo se u "valor" mais comum. O método bayesiano fornece resultados pouco melhores do que aqueles fornecidos pela estratégia simples e, embora o método da árvore de decisão use mais contexto e seja, portanto, mais preciso, ainda fornece resultados decepcionantes.

Em vez de tentar adivinhar valores de atributos desconhecidos, poderíamos tratar 'u nknow n' como um novo valor possível para cada atributo e lidar com ele da mesma f orma que outros valores. Isto pode levar a uma situação anômala, conforme mostrado no exemplo a seguir. Suponha que A seja um atributo com valores [A1, A2} e seja C uma coleção de objetos tais que

$$p_1 = 2 \quad p_2 = 2$$
$$n_1 = 2 \quad n_2 = 2$$

dando um valor de 1 bit para E(A). Agora seja A ' um atributo idêntico, exceto que um dos objetos com valor A1 de A tem um valor desconhecido de A '. A 'tem os valores { A ' 1, A ' 2, A ' 3= desconhecido }, então os valores correspondentes podem ser

$$p'_1 = 1 \quad p'_2 = 2 \quad p'_3 = 1$$
$$n'_1 = 2 \quad n'_2 = 2 \quad n'_3 = 0$$

resulting in a value of 0.84 bits for E(A′). In terms of the selection criterion developed earlier, A′ now seems to give a higher information gain than A. Thus, having unknown values may apparently increase the desirability of an attribute, a result entirely opposed to common sense. The conclusion is that treating 'unknown' as a separate value is not a solution to the problem.

One strategy which has been found to work well is as follows. Let A be an attribute with values $\{A_1, A_2, \ldots A_v\}$. For some collection C of objects, let the numbers of objects with value A_i of A be p_i and n_i, and let p_u and n_u denote the numbers of objects of class P and N respectively that have unknown values of A. When the information gain of attribute A is assessed, these objects with unknown values are distributed across the values of A in proportion to the relative frequency of these values in C. Thus the gain is assessed as if the true value of p_i were given by

$$p_i + p_u \cdot \text{ratio}_i$$

where

$$\text{ratio}_i = \frac{p_i + n_i}{\sum_i (p_i + n_i)}$$

and similarly for n_i. (This expression has the property that unknown values can only decrease the information gain of an attribute.) When an attribute has been chosen by the selection criterion, objects with unknown values of that attribute are discarded before forming decision trees for the subsets $\{C_i\}$.

The other half of the story is how unknown attribute values are dealt with during classification. Suppose that an object is being classified using a decision tree that wishes to branch on attribute A, but the object's value of attribute A is unknown. The correct procedure would take the branch corresponding to the real value A_i but, since this value is unknown, the only alternative is to explore all branches without forgetting that some are more probable than others.

Conceptually, suppose that, along with the object to be classified, we have been passed a *token* with some value T. In the situation above, each branch of A_i is then explored in turn, using a token of value

$$T \cdot \text{ratio}_i$$

i.e. the given token value is distributed across all possible values in proportion to the ratios above. The value passed to a branch may be distributed further by subsequent tests on other attributes for which this object has unknown values. Instead of a single path to a leaf, there may now be many, each qualified by its token value. These token values at the leaves are summed for each class, the result of the classification being

resultando em um valor de 0,84 bits para E(A'). Em termos do critério de selecção desenvolvido anteriormente, A' parece agora proporcionar um maior ganho de informação do que A. Assim, ter valores desconhecidos pode aparentemente aumentar a desejabilidade de um atributo, um resultado totalmente oposto ao bom senso. A conclusão é que tratar o " desconhecido" como um valor separado não é uma solução para o problema.

Uma estratégia que funcionou bem é a seguinte. Seja A um atributo com valores [A1, A2 . . . . Av}. Para alguma coleção C de objetos, sejam os números de objetos com valor Ai de A Pi e ni, e sejam pu e nu os números de objetos das classes P e N, respectivamente, que possuem valores desconhecidos de A. Quando o ganho de informação do atributo A é avaliado, esses objetos com valores desconhecidos são distribuídos pelos valores de A em proporção à frequência relativa desses valores em C. Assim, o ganho é avaliado como se o valor verdadeiro de Pi fosse dado por

$$p_i + p_u \cdot \text{ratio}_i$$

onde

$$\text{ratio}_i = \frac{p_i + n_i}{\sum_i (p_i + n_i)}$$

e da mesma forma para ni. (Esta expressão tem a propriedade de que valores desconhecidos só podem diminuir o ganho de informação de um atributo.) Quando um atributo foi escolhido. pelo critério de seleção, objetos com valores desconhecidos daquele atributo são descartados antes de formar árvores de decisão para os subconjuntos [Ci}.

A outra metade da história é como os valores de atributos desconhecidos são tratados durante a classificação. Suponha que um objeto esteja sendo classificado usando uma árvore de decisão que deseja ramificar no atributo A, mas o valor do atributo A do objeto é desconhecido. O procedimento correto seria pegar o ramo correspondente ao valor real Ai mas, como esse valor é desconhecido, a única alternativa é explorar todos os ramos sem esquecer que alguns são mais prováveis que outros.

Conceitualmente, suponha que, junto com o objeto a ser classificado, tenhamos recebido um token com algum valor T. Na situação acima, cada ramo de Ai é então explorado por sua vez, usando um token de valor

Razão T •i

ou seja, o valor do token fornecido é distribuído por todos os valores possíveis em proporção às proporções acima. O valor passado para uma ramificação pode ser distribuído posteriormente por testes subsequentes em outros atributos para os quais este objeto possui valores desconhecidos. Em vez de um único~caminho para uma folha, agora pode haver muitos, cada um qualificado por seu valor de token. Esses valores de token nas folhas são somados para cada classe, sendo o resultado da classificação

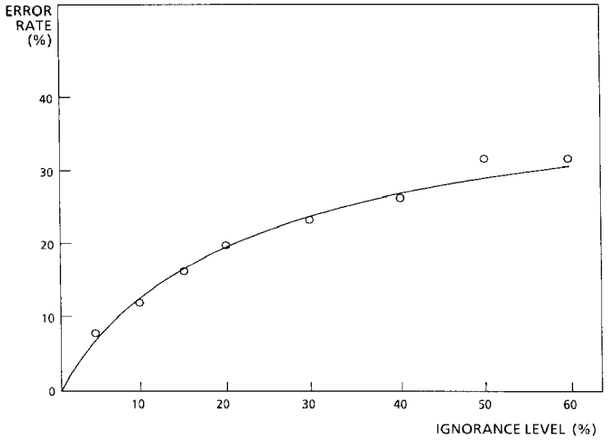


Figure 5. Error produced by unknown attribute values.

that class with the higher value. The distribution of values over the possible classes might also be used to compute a confidence level for the classification.

Straightforward though it may be, this procedure has been found to give a very graceful degradation as the incidence of unknown values increases. Figure 5 summarizes the results of an experiment on the now-familiar task with 551 objects and 39 attributes. Various 'ignorance levels' analogous to the earlier noise levels were explored, with twenty repititions at each level. For each run at an ignorance level of fn percent, a copy of the 551 objects was made, replacing each value of every attribute by 'unknown' with m percent probability. A decision tree for these (incomplete) objects was formed as above, and then used to classify a new copy of each object corrupted in the same way. The figure shows that the degradation of performance with ignorance level is gradual. In practice, of course, an ignorance level even as high as 10% is unlikely – this would correspond to an average of one value in every ten of the object's description being unknown. Even so, the decision tree produced from such a patchy training set correctly classifies nearly ninety percent of objects that also have unknown values. A much lower level of degradation is observed when an object with unknown values is classified using a correct decision tree.

This treatment has assumed that no information whatsoever is available regarding an unknown attribute. Catlett (1985) has taken this approach a stage further by allowing partial knowledge of an attribute value to be stated in Shafer notation (Garvey, Lowrance and Fischler, 1981). This notation permits probabilistic assertions to be made about any subset or subsets of the possible values of an attribute that an object might have.

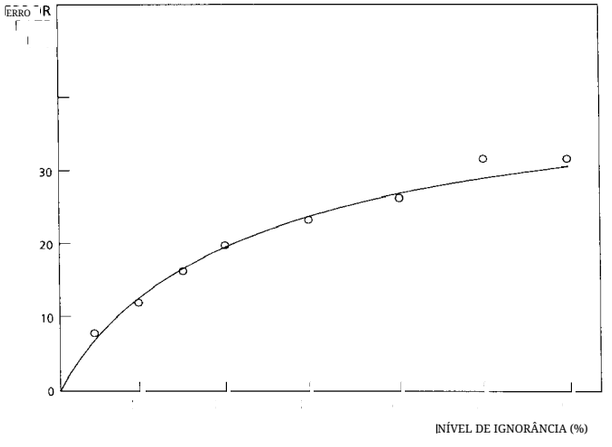


Figura 5. Erro produzido por valores de atributos desconhecidos.

aquela classe com o valor mais alto. A distribuição de valores pelas classes possíveis também pode ser usada para calcular um nível de confiança para a classificação.

Por mais simples que seja, descobriu-se que este procedimento proporciona uma degradação muito elegante à medida que aumenta a incidência de valores desconhecidos. A Figura 5 resume os resultados de um experimento na tarefa agora familiar com 551 objetos e 39 atributos. Foram explorados vários "níveis de ignorância" análogos aos níveis de ruído anteriores, com vinte repetições em cada nível. Para cada execução a um nível de ignorância de fn por cento, foi feita uma cópia dos 551 objetos, substituindo cada valor de cada atributo por 'desconhecido' com m por cento de probabilidade. Uma árvore de decisão para esses objetos (incompletos) foi formada como acima, e então usada para classificar uma nova cópia de cada objeto corrompido da mesma maneira. A figura mostra que a degradação do desempenho com o nível de ignorância é gradual. Na prática, é claro, um nível de ignorância tão alto quanto 10% é improvável - isso corresponderia a uma média de um valor em cada dez da descrição do objeto sendo desconhecido. Mesmo assim, a árvore de decisão produzida a partir de um conjunto de treinamento tão irregular classifica corretamente quase noventa por cento dos objetos que também possuem valores desconhecidos. Um nível de degradação muito menor é observado quando um objeto com valores desconhecidos é classificado usando uma árvore de decisão correta.

Este tratamento pressupõe que nenhuma informação está disponível sobre um atributo desconhecido. Catlett (1985) levou esta abordagem um passo adiante ao permitir que o conhecimento parcial de um valor de atributo fosse declarado na notação de Shafer (Garvey, Lowrance e Fischler, 1981). Esta notação permite que sejam feitas afirmações probabilísticas sobre qualquer subconjunto ou subconjuntos dos valores possíveis de um atributo que um objecto possa ter.

7. The selection criterion

Attention has recently been refocussed on the evaluation function for selecting the best attribute-based test to form the root of a decision tree. Recall that the criterion described earlier chooses the attribute that gains most information. In the course of their experiments, Bratko's group encountered a medical induction problem in which the attribute selected by the gain criterion ('age of patient', with nine value ranges) was judged by specialists to be less relevant than other attributes. This situation was also noted on other tasks, prompting Kononenko *et al* (1984) to suggest that the gain criterion tends to favor attributes with many values.

Analysis supports this finding. Let A be an attribute with values A_1, A_2, ... A_v and let A′ be an attribute formed from A by splitting one of the values into two. If the values of A were sufficiently fine for the induction task at hand, we would not expect this refinement to increase the usefulness of A. Rather, it might be anticipated that excessive fineness would tend to obscure structure in the training set so that A′ was in fact less useful than A. However, it can be proved that gain(A′) is greater than or equal to gain(A), being equal to it only when the proportions of the classes are the same for both subdivisions of the original value. In general, then, gain(A′) will exceed gain(A) with the result that the evaluation function of Section 4 will prefer A′ to A. By analogy, attributes with more values will tend to be preferred to attributes with fewer.

As another way of looking at the problem, let A be an attribute with random values and suppose that the set of possible values of A is large enough to make it unlikely that two objects in the training set have the same value for A. Such an attribute would have maximum information gain, so the gain criterion would select it as the root of the decision tree. This would be a singularly poor choice since the value of A, being random, contains no information pertinent to the class of objects in the training set.

ASSISTANT (Kononenko *et al*, 1984) solves this problem by requiring that all tests have only two outcomes. If we have an attribute A as before with v values A_1, A_2, ... A_v, the decision tree no longer has a branch for each possible value. Instead, a subset S of the values is chosen and the tree has two branches, one for all values in the set and one for the remainder. The information gain is then computed as if all values in S were amalgamated into one single attribute value and all remaining values into another. Using this selection criterion (the *subset* criterion), the test chosen for the root of the decision tree uses the attribute and subset of its values that maximizes the information gain. Kononenko *et al* report that this modification led to smaller decision trees with an improved classification performance. However, the trees were judged to be less intelligible to human beings, in agreement with a similar finding of Shepherd (1983).

Limiting decision trees to a binary format harks back to CLS, in which each test was of the form 'attribute A has value A_i', with two branches corresponding to true and false. This is clearly a special case of the test implemented in ASSISTANT, which

7. O critério de seleção

A atenção foi recentemente redirecionada para a função de avaliação para selecionar o " melhor teste baseado em atributos para formar a raiz de uma árvore de decisão. Lembre-se de que o critério descrito anteriormente escolhe o atributo que obtém mais informações. No decorrer de seus experimentos, o grupo de Bratko encontrou um problema de indução médica em que o atributo selecionado pelo critério de ganho ('idade do paciente', com nove faixas de valores) foi julgado por especialistas como menos relevante do que outros atributos. Essa situação também foi observada em o utras tarefas, levando Kononenko et al (1984) para sugerir que o critério de ganho tende a favorecer atributos com muitos valores.

A análise apóia esta descoberta• Seja A um atributo com valores A~, A2 .... Av e seja A ' um atributo formado a partir de A dividindo um dos valores em dois. Se os valores de A fossem suficientemente finos para a tarefa de indução em questão, não esperaríamos que este refinamento aumentasse a utilidade de A. Em vez disso, poderia ser antecipado que a finura excessiva tenderia a obscurecer a estrutura no conjunto de treinamento, de modo que A' fosse de fato menos útil que A. No entanto, pode-se provar que o ganho(A') é maior ou igual ao ganho(A), sendo igual a ele apenas quando as proporções das classes são as mesmas para ambas as subdivisões do valor original. Em geral, então, o ganho(A') excederá o ganho(A) com o resultado de que a função de avaliação da Secção 4 preferirá A' a A. Por analogia, atributos com mais valores tenderão a ser preferidos a atributos com menos.

Como outra maneira de encarar o problema, seja A um atributo com valores aleatórios e suponha que o conjunto de valores possíveis de A seja grande o suficiente para tornar improvável que dois objetos no conjunto de treinamento tenham o mesmo valor para A. Tal atributo teria ganho máximo de informação, então o critério de ganho o selecionaria como a raiz da árvore de decisão. Esta seria uma escolha singularmente ruim, uma vez que o valor de A, sendo aleatório, não contém nenhuma informação pertinente à classe de objetos no conjunto de treinamento.

AS S IST ANT (Kononenko et al, 1984) resolve esse problema exigindo que todos os testes~ tenham apenas dois resultados. Se tivermos um atributo A como antes com v valores A1, A2,

•. . Av, a árvore de decisão não possui mais uma ramificação para cada valor possível. Em vez disso, um subconjunto S de valores é escolhido e a árvore tem dois ramos, um para todos os valores do conjunto e outro para o restante. O ganho de informação é então calculado como se todos os valores em S fossem amalgamados em um único valor de atributo e todos os valores restantes em outro. Utilizando este crit ério de seleção (critério do subconjunto), o teste escolhido para a raiz da árvore de decisão utiliza o atributo e subconjunto de seus valores que maximiza o ganho de informação. Kononenko et al relatam que esta modificação levou a árvores de decisão menores com melhor desempenho de classificação. No entanto, as árvores foram consideradas menos inteligíveis para os seres humanos, de acordo com uma conclusão semelhante de Shepherd (1983).

Limitar as árvores de decisão a um formato binário remonta ao CLS, em que cada teste tinha a forma 'atributo A tem valor Ai', com duas ramificações correspondendo a verdadeiro e falso• Este é claramente um caso especial do teste implementado no ASSISTANT, que

permits a set of values, rather than a single value, to be distinguished from the others.

It is also worth noting that the method of dealing with attributes having continuous values follows the same binary approach. Let A be such an attribute and suppose that the distinct values of A that occur in C are sorted to give the sequence $V_1, V_2, \ldots, V_k$. Each pair of values V_i, V_{i+1} suggests a possible threshold

$$\frac{V_i + V_{i+1}}{2}$$

that divides the objects of C into two subsets, those with a value of A above and below the threshold respectively. The information gain of this division can then be investigated as above.

If all tests must be binary, there can be no bias in favor of attributes with large numbers of values. It could be argued, however, that ASSISTANT's remedy has undesirable side-effects that have to be taken into account. First, it can lead to decision trees that are even more unintelligible to human experts than is ordinarily the case, with unrelated attribute values being grouped together and with multiple tests on the same attribute.

More importantly, the subset criterion can require a large increase in computation. An attribute A with v values has 2^v value subsets and, when trivial and symmetric subsets are removed, there are still $2^{v-1}-1$ different ways of specifying the distinguished subset of attribute values. The information gain realized with each of these must be investigated, so a single attribute with v values has a computational requirement similar to $2^{v-1}-1$ binary attributes. This is not of particular consequence if v is small, but the approach would appear infeasible for an attribute with 20 values.

Another method of overcoming the bias is as follows. Consider again our training set containing p and n objects of class P and N respectively. As before, let attribute A have values $A_1, A_2, \ldots A_v$ and let the numbers of objects with value A_i of attribute A be p_i and n_i respectively. Enquiring about the value of attribute A itself gives rise to information, which can be expressed as

$$IV(A) = -\sum_{i=1}^{v} \frac{p_i+n_i}{p+n} \log_2 \frac{p_i+n_i}{p+n}$$

IV(A) thus measures the information content of the answer to the question, 'What is the value of attribute A?' As discussed earlier, gain(A) measures the reduction in the information requirement for a classification rule if the decision tree uses attribute A as a root. Ideally, as much as possible of the information provided by determining the value of an attribute should be useful for classification purposes or, equivalently, as little as possible should be 'wasted'. A good choice of attribute would then be one for which the ratio

permite que um conjunto de valores, em vez de um único valor, seja distinguido dos outros.

Também é importante notar que o método de lidar com atributos com valores contínuos segue a mesma abordagem binária. Seja A um tal atributo e suponha que os valores distintos de A que ocorrem em C sejam ordenados para fornecer a sequência Va, V2. . . . , Vk. Cada par de valores Vi, Vi + l sugere um possível limite

$$\frac{V_i + V_{i+1}}{2}$$

que divide os objetos de C em dois subconjuntos, aqueles com valor de A acima e abaixo do limite, respectivamente. O ganho de informação desta divisão pode então ser investigado como acima.

Se todos os testes devem ser binários, não pode haver preconceito em favor de atributos com grande número de valores. Pode-se argumentar, entretanto, que o remédio da A S I S T A N T tem efeitos colaterais indesejáveis que devem ser levados em consideração. Primeiro, pode levar a árvores de decisão que são ainda mais ininteligíveis para os especialistas humanos do que normalmente acontece, com valores de atributos não relacionados sendo agrupados e com múltiplos testes no mesmo atributo.

Mais importante ainda, o critério do subconjunto pode exigir um grande aumento na computação. Um atributo A com v valores possui 2 v subconjuntos de valores e, quando subconjuntos triviais e simétricos são removidos, ainda existem 2 v - l - 1 maneiras diferentes de especificar o subconjunto distinto de valores de atributos. O ganho de informação realizado com cada um deles deve ser investigado, portanto, um único atributo com valores v possui um requisito computacional semelhante a 2 atributos binários v-1 _ 1. Isso não tem consequências específicas se v for pequeno, mas a abordagem pareceria inviável para um atributo com 20 valores.

Outro método para superar o preconceito é o seguinte. Considere novamente nosso conjunto de treinamento contendo p e n objetos das classes P e N, respectivamente. Como antes, deixe o atributo A ter valores A1, A2. . . . Av e sejam os números de objetos com valor Ai do atributo A pl e nl respectivamente. Indagar sobre o valor do próprio atributo A dá origem a informações, que podem ser expressas como

$$IV(A) = -\sum_{i=1}^{v} \frac{p_i+n_i}{p+n} \log_2 \frac{p_i+n_i}{p+n}$$

IV(A) mede assim o conteúdo informativo da resposta à pergunta: 'Qual é o valor do atributo A?' Conforme discutido anteriormente, o ganho(A) mede a redução na exigência de informação para uma regra de classificação se a árvore de decisão usar o atributo A como raiz. Idealmente, o máximo possível da informação fornecida pela determinação do valor de um atributo deveria ser útil para fins de classificação ou, equivalentemente, o mínimo possível deveria ser "desperdiçado". Uma boa escolha de atributo seria então aquela para a qual a proporção

gain(A) / IV(A)

is as large as possible. This ratio, however, may not always be defined – IV(A) may be zero – or it may tend to favor attributes for which IV(A) is very small. The *gain ratio* criterion selects, from among those attributes with an average-or-better gain, the attribute that maximizes the above ratio.

This can be illustrated by returning to the example based on the training set of Table 1. The information gain of the four attributes is given in Section 4 as

gain(outlook) = 0.246 bits
gain(temperature) = 0.029 bits
gain(humidity) = 0.151 bits
gain(windy) = 0.048 bits

Of these, only outlook and humidity have above-average gain. For the outlook attribute, five objects in the training set have the value sunny, four have overcast and five have rain. The information obtained by determining the value of the outlook attribute is therefore

$$\text{IV(outlook)} = -\frac{5}{14}\log_2\frac{5}{14} - \frac{4}{14}\log_2\frac{4}{14} - \frac{5}{14}\log_2\frac{5}{14} = 1.578 \text{ bits}$$

Similarly,

$$\text{IV(humidity)} = -\frac{7}{14}\log_2\frac{7}{14} - \frac{7}{14}\log_2\frac{7}{14} = 1 \text{ bit}$$

So,

gain ratio(outlook) = 0.246 / 1.578 = 0.156
gain ratio(humidity) = 0.151 / 1.000 = 0.151

The gain ratio criterion would therefore still select the outlook attribute for the root of the decision tree, although its superiority over the humidity attribute is now much reduced.

The various selection criteria have been compared empirically in a series of experiments (Quinlan, 1985b). When all attributes are binary, the gain ratio criterion has been found to give considerably smaller decision trees: for the 551-object task, it produces a tree of 143 nodes compared to the smallest previously-known tree of 175 nodes. When the task includes attributes with large numbers of values, the subset criterion gives smaller decision trees that also have better predictive performance, but can require much more computation. However, when these many-valued attributes are augmented by redundant attributes which contain the same information at a

ganho(A) / IV(A)

é o maior possível. Este rácio, no entanto, nem sempre pode ser definido - IV(A) pode s er zero - ou pode tender a favorecer atributos para os quais IV(A) é muito pequeno. O critério da razão de ganho seleciona, dentre os atributos com ganho médio ou melhor, o atributo que maximiza a relação acima.

Isto pode ser ilustrado retornando ao exemplo baseado no conjunto de treinamento da Tabela 1. O ganho de informação dos quatro atributos é dado na Seção 4 como

ganho (perspectiva) = 0
ganho de 0,246 bits (te
mperatura) = 0 ganho de 0,029
bits (umidade) = 0 ganho de
0,151 bits (ventoso) = 0 0,048 bits

Destes, apenas perspectiva e umidade apresentam ganho acima da média. Para o atributo Outlook, cinco objetos no conjunto de treinamento têm o valor ensolarado, quatro têm nublado e cinco têm chuva. A informação obtida através da determinação do valor do atributo da perspectiva é, portanto,

$$\text{IV(outlook)} = -\frac{5}{14}\log_2\frac{5}{14} - \frac{4}{14}\log_2\frac{4}{14} - \frac{5}{14}\log_2\frac{5}{14} = 1.578 \text{ bits}$$

De forma
similar,

$$\text{IV(humidity)} = -\frac{7}{14}\log_2\frac{7}{14} - \frac{7}{14}\log_2\frac{7}{14} = 1 \text{ bit}$$

gain ratio(outlook) = 0.246 / 1.578 = 0.156
gain ratio(humidity) = 0.151 / 1.000 = 0.151

O critério da razão de ganho, portanto, ainda selecionaria o atributo de perspectiva para a raiz da árvore de decisão, embora a sua superioridade sobre o atributo de umidade seja agora muito reduzida.

Os vários critérios de selecção foram comparados empiricamente numa série de experiências (Quinlan, 1985b). Quando todos os atributos são binários, descobriu-se que o critério da razão de ganho fornece árvores de decisão consideravelmente menores: para a tarefa de 551 objetos, ele produz uma árvore de 143 nós em comparação com a menor árvore anteriormente conhecida de 175 nós. Quando a tarefa inclui atributos com grande número de valores, o critério do subconjunto fornece árvores de decisão menores que também apresentam melhor desempenho preditivo, mas podem exigir muito mais computação. No entanto, quando esses atributos de muitos valores são aumentados por atributos redundantes que contêm a mesma informação em um nível

lower level of detail, the gain ratio criterion gives decision trees with the greatest predictive accuracy. All in all, these experiments suggest that the gain ratio criterion does pick a good attribute for the root of the tree. Testing an attribute with many values, however, will fragment the training set C into very small subsets $\{C_i\}$ and the decision trees for these subsets may then have poor predictive accuracy. In such cases, some mechanism such as value subsets or redundant attributes is needed to prevent excessive fragmentation.

The three criteria discussed here are all information-based, but there is no reason to suspect that this is the only possible basis for such criteria. Recall that the modifications to deal with noise barred an attribute from being used in the decision tree unless it could be shown to be relevant to the class of objects in the training set. For any attribute A, the value of the statistic presented in Section 5, together with the number v of possible values of A, determines the confidence with which we can reject the null hypothesis that an object's value of A is irrelevant to its class. Hart (1985) has proposed that this same test could function directly as a selection criterion: simply pick the attribute for which this confidence level is highest. This measure takes explicit account of the number of values of an attribute and so may not exhibit bias. Hart notes, however, that the chi-square test is valid only when the expected values of p'_i and n'_i are uniformly larger than four. This condition could be violated by a set C of objects either when C is small or when few objects in C have a particular value of some attribute, and it is not clear how such sets would be handled. No empirical results with this approach are yet available.

8. Conclusion

The aim of this paper has been to demonstrate that the technology for building decision trees from examples is fairly robust. Current commercial systems are powerful tools that have achieved noteworthy successes. The groundwork has been done for advances that will permit such tools to deal even with noisy, incomplete data typical of advanced real-world applications. Work is continuing at several centers to improve the performance of the underlying algorithms.

Two examples of contemporary research give some pointers to the directions in which the field is moving. While decision trees generated by the above systems are fast to execute and can be very accurate, they leave much to be desired as representations of knowledge. Experts who are shown such trees for classification tasks in their own domain often can identify little familiar material. It is this lack of familiarity (and perhaps an underlying lack of modularity) that is the chief obstacle to the use of induction for building large expert systems. Recent work by Shapiro (1983) offers a possible solution to this problem. In his approach, called *Structured Induction*, a rule-formation task is tackled in the same style as structured programming. The task is solved in terms of a collection of notional super-attributes, after which the subtasks

nível de detalhe mais baixo, o critério da taxa de ganho fornece árvores de decisão co m a maior precisão preditiva. Em suma, estas experiências sugerem que o critério da razão de ganho escolhe um bom atributo para a raiz da árvore. Testar um atributo com muitos valores, entretanto, fragmentará o conjunto de treinamento C em subconjuntos muito pequenos [Ci} e as árvores de decisão para esses subconjuntos poderão então ter baixa precisão preditiva. Nesses casos, é necessário algum mecanismo, como subconjuntos de valores ou atributos redundantes, para evitar fragmentação excessiva.

Os três critérios aqui discutidos são todos baseados em informação, mas não há razão para suspeitar que esta seja a única base possível para tais critérios. Lembre-se de que as modificações para lidar com o ruído impediram que um atributo fosse usado na árvore de decisão, a menos que pudesse ser demonstrado que era relevante para a classe de objetos no conjunto de treinamento. Para qualquer atributo A, o valor da estatística apresentada na Seção 5, juntamente com o número v de valores possíveis de A, determina a confiança com a qual podemos rejeitar a hipótese nula de que o valor de A de um objeto é irrelevante para sua classe. Hart (1985) propôs que esse mesmo teste poderia funcionar diretamente como critério de seleção: basta escolher o atributo para o qual esse nível de confiança é mais alto. Esta medida leva explicitamente em conta o número de valores de um atributo e, portanto, não pode apresentar viés. Hart observa, entretanto, que o teste do qui-quadrado é válido apenas quando os valores esperados de p ' i e n '] são uniformemente maiores que quatro. Esta condição pode ser violada por um conjunto C de objetos quando C é pequeno ou quando poucos objetos em C possuem um valor particular de algum atributo, e não está claro como tais conjuntos seriam tratados. Ainda não estão disponíveis resultados empíricos com esta abordage m.

"8. Conclusão

O objetivo deste artigo foi demonstrar que a tecnologia para construir árvores de decisão a partir de exemplos é bastante robusta. Os sistemas comerciais atuais são ferramentas poderosas que alcançaram sucessos notáveis. A base foi feita para avanços que permitirão que tais ferramentas lidem até mesmo com dados ruidosos e incompletos, típicos de aplicações avançadas do mundo real. O trabalho continua em vários centros para melhorar o desempenho dos algoritmos subjacentes.

Dois exemplos de pesquisa contemporânea fornecem algumas indicações sobre as direções em que o campo está se movendo. Embora as árvores de decisão geradas pelos sistemas acima sejam de execução rápida e possam ser muito precisas, elas deixam muito a desejar como representações do conhecimento. Os especialistas que vêem essas árvores para tarefas de classificação em seu próprio domínio geralmente conseguem identificar pouco material familiar. É esta falta de familiaridade (e talvez uma falta subjacente de modularidade) que é o principal obstáculo ao uso da indução na construção de grandes sistemas especialistas. O trabalho recente de Shapiro (1983) oferece uma solução possível para este problema. Em sua abordagem, chamada Indução Estruturada, uma tarefa de formação de regras é abordada no mesmo estilo da programação estruturada. A tarefa é resolvida em termos de uma coleção de superatributos nocionais, após os quais as subtarefas

of inducing classification rules to find the values of the super-attributes are approached in the same top-down fashion. In one classification problem studied, this method reduced a totally opaque, large decision tree to a hierarchy of nine small decision trees, each of which 'made sense' to an expert.

ID3 allows only two classes for any induction task, although this restriction has been removed in most later systems. Consider, however, the task of developing a rule from a given set of examples for classifying an animal as a monkey, giraffe, elephant, horse, etc. A single decision tree could be produced in which these various classes appeared as leaves. An alternative approach taken by systems such as INDUCE (Michalski, 1980) would produce a collection of classification rules, one to discriminate monkeys from non-monkeys, another to discriminate giraffes from non-giraffes, and so on. Which approach is better? In a private communication, Marcel Shoppers has set out an argument showing that the latter can be expected to give more accurate classification of objects that were not in the training set. The multi-tree approach has some associated problems – the separate decision trees may classify an animal as both a monkey and a giraffe, or fail to classify it as anything, for example – but if these can be sorted out, this approach may lead to techniques for building more reliable decision trees.

Acknowledgements

It is a pleasure to acknowledge the stimulus and suggestions provided over many years by Donald Michie, who continues to play a central role in the development of this methodology. Ivan Bratko, Igor Kononenko, Igor Mosetic and other members of Bratko's group have also been responsible for many insights and constructive criticisms. I have benefited from numerous discussions with Ryszard Michalski, Alen Shapiro, Jason Catlett and other colleagues. I am particularly grateful to Pat Langley for his careful reading of the paper in draft form and for the many improvements he recommended.

References

Buchanan, B.G., & Mitchell, T.M. (1978). Model-directed learning of production rules. In D.A. Waterman, F. Hayes-Roth (Eds.), *Pattern directed inference systems*. Academic Press.

Carbonell, J.G., Michalski, R.S., & Mitchell, T.M. (1983). An overview of machine learning, In R.S. Michalski, J.G. Carbonell and T.M. Mitchell, (Eds.), *Machine learning: An artificial intelligence approach*. Palo Alto: Tioga Publishing Company.

Catlett, J. (1985). *Induction using the shafer representation* (Technical report). Basser Department of Computer Science, University of Sydney, Australia.

Dechter, R., & Michie, D. (1985). *Structured induction of plans and programs* (Technical report). IBM Scientific Center, Los Angeles, CA.

de induzir regras de classificação para encontrar os valores dos superatributos são abordados da mesma maneira de cima para baixo. Num problema de classificação estudado, este método reduziu uma grande árvore de decisão totalmente opaca a uma hierarquia de nove pequenas árvores de decisão, cada uma das quais "fazia sentido" para um especialista.

ID3 permite apenas duas classes para qualquer tarefa de indução, embora esta restrição tenha sido removida na maioria dos sistemas posteriores. Consideremos, contudo, a tarefa de desenvolver uma regra a partir de um dado conjunto de exemplos para classificar um animal como macaco, girafa, elefante, cavalo, etc. Uma única árvore de decisão poderia ser produzida na qual estas várias classes aparecessem como folhas. Uma abordagem alternativa adoptada por sistemas como o INDUCE (Michalski, 1980) produziria uma colecção de regras de classificação, uma para discriminar macacos de não-macacos, outra para discriminar girafas de não-girafas, e assim por diante. Qual abordagem é melhor? Numa comunicação privada, Marcel Shoppers apresentou um argumento mostrando que se pode esperar que este último forneça uma classificação mais precisa de objetos que não estavam no conjunto de treinamento. A abordagem de múltiplas árvores tem alguns problemas associados - as árvores de decisão separadas podem classificar um animal tanto como macaco como como girafa, ou não conseguirem classificá-lo como qualquer coisa, por exemplo - mas se estes puderem ser resolvidos, esta abordagem pode levar a técnicas para a construção de árvores de decisão mais fiáveis.

Agradecimentos

É um prazer reconhecer o estímulo e as sugestões fornecidas ao longo de muitos anos por Donald Michie, que continua a desempenhar um papel central no desenvolvimento desta metodologia. Ivan Bratko, Igor Kononenko, Igor Mosetic e outros membros do grupo de Bratko também foram responsáveis por muitos insights e críticas construtivas. Beneficiei-me de inúmeras discussões com Ryszard Michalski, Alen Shapiro, Jason Catlett e outros colegas. Estou particularmente grato a Pat Langley" pela sua leitura cuidadosa do documento em forma de rascunho e pelas muitas melhorias que recomendou.

Referências

Buchanan, BG e Mitchell, TM. (1978). Aprendizagem dirigida por modelo de regras de produção. Em D.A. Waterman, F. Hayes-Roth (Eds.), Sistemas de inferência dirigidos por padrões. Imprensa Acadêmica. Carbonell, JG, Michalski, RS e Mitchell, TM. (1983). Uma visão geral do aprendizado de máquina, em R.S. Michalski, J.G. Carbonell e T.M. Mitchell, (Eds.), Aprendizado de máquina: uma abordagem de inteligência artificial. Palo Alto: Editora Tioga. Catlett, J. (1985). Indução utilizando a representação shafer (Relatório técnico). Departamento Basser de Ciência da Computação, Universidade de Sydney, Austrália. Dechter, R. e Michie, D. (1985). Indução estruturada de planos e programas (Relatório técnico). Centro Científico IBM, Los Angeles, CA.

Feigenbaum, E.A., & Simon, H.A. (1963). Performance of a reading task by an elementary perceiving and memorizing program, *Behavioral Science, 8*.

Feigenbaum, E.A. (1981). Expert systems in the 1980s. In A. Bond (Ed.), *State of the art report on machine intelligence.* Maidenhead: Pergamon-Infotech.

Garvey, T.D., Lowrance, J.D., & Fischler, M.A. (1981). An inference technique for integrating knowledge from disparate sources. *Proceedings of the Seventh International Joint Conference on Artificial Intelligence.* Vancouver, B.C., Canada: Morgan Kaufmann.

Hart, A.E. (1985). Experience in the use of an inductive system in knowledge engineering. In M.A. Bramer (Ed.), *Research and development in expert systems.* Cambridge University Press.

Hogg, R.V., & Craig, A.T. (1970). *Introduction to mathematical statistics.* London: Collier-Macmillan.

Hunt, E.B. (1962). *Concept learning: An information processing problem.* New York: Wiley.

Hunt, E.B., Marin, J., & Stone, P.J. (1966). *Experiments in induction.* New York: Academic Press.

Kononenko, I., Bratko, I., & Roskar, E. (1984). *Experiments in automatic learning of medical diagnostic rules* (Technical report). Jozef Stefan Institute, Ljubljana, Yugoslavia.

Langley, P., Bradshaw, G.L., & Simon, H.A. (1983). Rediscovering chemistry with the BACON system. In R.S. Michalski, J.G. Carbonell and T.M. Mitchell (Eds.), *Machine learning: An artificial intelligence approach.* Palo Alto: Tioga Publishing Company.

Michalski, R.S (1980). Pattern recognition as rule-guided inductive inference. *IEEE Transactions on Pattern Analysis and Machine Intelligence 2.*

Michalski, R.S., & Stepp, R.E. (1983). Learning from observation: conceptual clustering. In R.S. Michalski, J.G. Carbonell & T.M. Mitchell (Eds.), *Machine learning: An artificial intelligence approach.* Palo Alto: Tioga Publishing Company.

Michie, D. (1982). Experiments on the mechanisation of game-learning 2 – Rule-based learning and the human window. *Computer Journal 25.*

Michie, D. (1983). Inductive rule generation in the context of the Fifth Generation. *Proceedings of the Second International Machine Learning Workshop.* University of Illinois at Urbana-Champaign.

Michie, D. (1985). Current developments in Artificial Intelligence and Expert Systems. In *International Handbook of Information Technology and Automated Office Systems.* Elsevier.

Nilsson, N.J. (1965). *Learning machines*, New York: McGraw-Hill.

O'Keefe, R.A. (1983). Concept formation from very large training sets. In *Proceedings of the Eighth International Joint Conference on Artificial Intelligence.* Karlsruhe, West Germany: Morgan Kaufmann.

Patterson, A., & Niblett, T. (1983). *ACLS user manual.* Glasgow: Intelligent Terminals Ltd.

Pearl, J. (1978a). Entropy, information and rational decisions (Technical report). Cognitive Systems Laboratory, University of California, Los Angeles.

Pearl, J. (1978b). On the connection between the complexity and credibility of inferred models. *International Journal of General Systems, 4.*

Quinlan, J.R. (1969). A task-independent experience gathering scheme for a problem solver. *Proceedings of the First International Joint Conference on Artificial Intelligence.* Washington, D.C.: Morgan Kaufmann.

Quinlan, J.R. (1979). Discovering rules by induction from large collections of examples. In D. Michie (Ed.), *Expert systems in the micro electronic age.* Edinburgh University Press.

Quinlan, J.R. (1982). Semi-autonomous acquisition of pattern-based knowledge. In J.E. Hayes, D. Michie & Y-H. Pao (Eds.), *Machine intelligence 10.* Chichester: Ellis Horwood.

Quinlan, J.R. (1983a). Learning efficient classification procedures and their application to chess endgames. In R.S. Michalski, J.G. Carbonell & T.M. Mitchell, (Eds.), *Machine learning: An artificial intelligence approach.* Palo Alto: Tioga Publishing Company.

Quinlan, J.R. (1983b). Learning from noisy data, *Proceedings of the Second International Machine Learning Workshop.* University of Illinois at Urbana-Champaign.

Quinlan, J.R. (1985a). The effect of noise on concept learning. In R.S. Michalski, J.G. Carbonell & T.M.

Feigenbaum, EA e Simon, HA. (1963). Desempenho de uma tarefa de leitura por meio de um programa elementar de percepção e memorização, Behavioral Science, 8. Feigenbaum, E.A. (1981). Sistemas especialistas na década de 1980. Em A. Bond (Ed.), Relatório do estado da arte sobre inteligência de máquina. Maidenhead: Pergamon-Infoteeh. Garvey, TD, Lowrance, JD e Fischler, MA (1981). Uma técnica de inferência para integrar conhecimento de fontes díspares. Anais da Sétima Conferência Conjunta Internacional sobre Inteligência Artificial. Vancouver, BC, Canadá: Morgan Kaufmann. Hart, AE (1985). Experiência na utilização de sistema indutivo em engenharia do conhecimento. Em MA Bramer (Ed.), Pesquisa e desenvolvimento em sistemas especialistas. I mprensa da Universidade de Cambridge. Hogg, RV e Craig, AT. (1970). Introdução à estatística matemática. Londres: Collier-Macmillan. Hunt, E. B. (1962). Aprendizagem de conceitos: um problema de processamento de informações. Nova York: Wiley. Hunt, EB, Marin, J. e Stone, PJ (1966). Experimentos em indução. Nova York: Academic Press. Kononenko, I., Bratko, I. e Roskar, E. (1984). Experimentos de aprendizagem automática de regras de diagnóstico médico (Relatório técnico). Instituto Jozef Stefan, Liubliana, Iugoslávia. Langley, P., Bradshaw, GL e Simon, HA. (1983). Redescobrindo a química com o sistema BACON. Em R.S. Michalski, J.G. Carbonell e T.M. Mitchell (Eds.), Aprendizado de máquina: uma abordagem de inteligência artificial. Palo Alto: Editora Tioga. Michalski, RS (1980). Reconhecimento de padrões como inferência indutiva guiada por regras. 1EEE Transações em Análise de Padrões e Inteligência de Máquina 2. Michalski, R.S., & Stepp, R.E. (1983). Aprendendo com a observação: agrupamento conceitual. Em R.S. Michalski, J.G. CarboneU & T.M. Mitchell (Eds.), Aprendizado de máquina: uma abordagem de inteligência artificial. Palo Alto: Editora Tioga. Michie, D. (1982). Experimentos sobre a mecanização da aprendizagem por jogos 2 - Aprendizagem baseada em regras e a janela humana. Diário de Computador 25. Michie, D. (1983). Geração de regras indutivas no contexto da Quinta Geração. Anais do Segundo Workshop Internacional de Aprendizado de Máquina. Universidade de Illinois em Urbana-Champaign. Michie, D. (1985). Desenvolvimentos atuais em Inteligência Artificial e Sistemas Especialistas. No Manual Internacional de Tecnologia da Informação e Sistemas de Escritório Automatizados. Elsevier. Nilsson, NJ (1965). Máquinas de aprendizagem, Nova York: McGraw-Hill. O'Keefe, R.A. (1983). Formação de conceitos a partir de conjuntos de treinamento muito grandes. Nos Anais da Oitava Conferência Conjunta Internacional sobre Inteligência Artificial. Karlsruhe, Alemanha Ocidental: Morgan Kaufmann. Patterson, A. e Niblett, T. (1983). Manual do usuário ACLS. Glasgow: Intelligent Terminals Ltd. Pearl, J. (1978a). Entropia, informação e decisões racionais (Relatório técnico). Laboratório de Sistemas Cognitivos, Universidade da Califórnia, Los Angeles. Pérola, J. (1978b). Sobre a ligação entre a complexidade e a credibilidade dos modelos inferidos. International Journal of General Systems, 4. Quinlan, JR (1969). Um esquema de coleta de experiência independente de tarefas para um solucionador de problemas. Anais da Primeira Conferência Conjunta Internacional sobre Inteligência Artificial. Washington, DC: Morgan Kaufmann. Quinlan, JR (1979). Descobrir regras por indução a partir de grandes coleções de exemplos. Em D. Michie (Ed.), Sistemas especialistas na era microeletrônica. Imprensa da Universidade de Edimburgo. Quinlan, JR (1982). Aquisição semiautônoma de conhecimento baseado em padrões. Em JE Hayes, D. Michie & YH. Pao (Eds.), Inteligência de máquina 10. Chichester: Ellis Horwood. Quinlan, JR (1983a). Aprender procedimentos de classificação eficientes e sua aplicação em jogos finais de xadrez. Em R.S. Michalski, J.G. Carbonell & T.M. Mitchell, (Eds.), Aprendizado de máquina: uma abordagem de inteligência artificial. Palo Alto: Editora Tioga. Quinlan, JR (1983b). Aprendendo com dados ruidosos, Anais do Segundo Workshop Internacional de Aprendizado de Máquina. Universidade de Illinois em Urbana-Champaign. Quinlan, JR (1985a). O efeito do ruído na aprendizagem de conceitos. Em R.S. Michalski, J.G. Carbonell & T.M.

Mitchell (Eds.), *Machine learning*. Los Altos: Morgan Kaufmann (in press).
Quinlan, J.R. (1985b). Decision trees and multi-valued attributes. In J.E. Hayes & D. Michie (Eds.), *Machine intelligence 11*. Oxford University Press (in press).
Sammut, C.A. (1985). Concept development for expert system knowledge bases. *Australian Computer Journal 17*.
Samuel, A. (1967). Some studies in machine learning using the game of checkers II: Recent progress. *IBM J. Research and Development 11*.
Shapiro, A. (1983). *The role of structured induction in expert systems*. Ph.D. Thesis, University of Edinburgh.
Shepherd, B.A. (1983). An appraisal of a decision-tree approach to image classification. *Proceedings of the Eighth International Joint Conference on Artificial Intelligence*. Karlsruhe, West Germany: Morgan Kaufmann.
Winston, P.H. (1975). Learning structural descriptions from examples. In P.H. Winston (Ed.), *The psychology of computer vision*. McGraw-Hill.

Mitchell (Eds.), Aprendizado de máquina. Los Altos: Morgan Kaufmann (no prelo). Quinlan, JR (1985b). Árvores de decisão e atributos multivalorados. Em JE Hayes & D. Michie (Eds.), Machine intelligence 11. Oxford University Press (no prelo). Sammut, C. A. (1985). Desenvolvimento de conceito para bases de conhecimento de sistemas especialistas. Australian Computer Journal 17. Samuel, A. (1967). Alguns estudos em aprendizado de máquina utilizando o jogo de damas II: Progressos recentes. IBM J. Pesquisa e Desenvolvimento 11. Shapiro, A. (1983). O papel da indução estruturada em sistemas especialistas. Ph.D. Tese, Universidade de Edimburgo. Pastor, B.A. (1983). Uma avaliação de uma abordagem de árvore de decisão para classificação de imagens. Anais da Oitava Conferência Conjunta Internacional sobre Inteligência Artificial. Karlsruhe, Alemanha Ocidental: Morgan Kaufmann. Winston, P.H. (1975). Aprender descrições estruturais a partir de exemplos. Em P.H. Winston (Ed.), A psicologia da visão computacional. McGraw-Hill.